

AI, Photography and the Battle for Reality



Cole Werner



Evelyn Truong



Owen Ricker



Emily Hansson



Morgan Meng

[AI Generated](#)





Agenda



Introduction to the Dataset

Topic

Dataset Metrics

Pre-Observations



Kaggle AI Generated Images vs Real Images

Cash Bowman | <https://www.kaggle.com/datasets/cashbowman/ai-generated-images-vs-real-images/data>

1

Image Collection

Web-scraped images for both real and AI-generated landscapes, paintings, and digital imagery

2

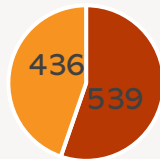
Data Packaging

Provided 2 folders of jpeg/png/jpg images of varying dimensions for “AiArtData” and “RealArt”

❖ Avg WxH: 1200x950

3

Class Distribution



■ AI Generated ■ Real Images

4

Applications

- ❖ Spotting deepfakes
- ❖ Protecting digital identities
- ❖ Academic integrity
- ❖ Removing AI images from being used for training of other AI models
- ❖ Filter AI generated content on social media (i.e. content moderation)

Dataset Image Samples

AI Generated

Drawings



Realism



Landscapes



Abstract



Real

Famous Paintings



Photographs



Landscapes



Abstract Realism

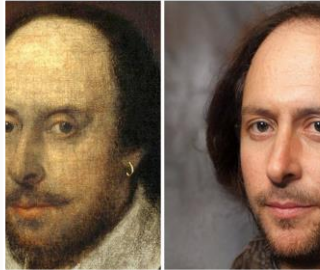


Potential Challenges

AI



Composite Images containing more than just a single image and irrelevant textual information



Composite images where the AI and real image are side-by-side comparisons



Images that look computer generated in the first place

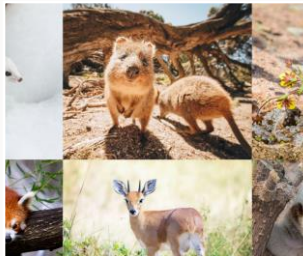


Outlier images where there's not a lot of that per art-type category (i.e. person, real)



Some images are much larger (ex. 3733 x 2133) and others are smaller (ex. 626 x 358)

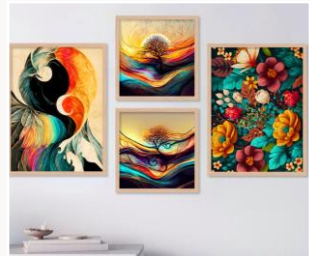
Real



Composite Images containing more than just a single image and irrelevant textual information



Photos with poor resolution that may be harder to distinguish features



Images that look computer generated in the first place like this stock images type photo



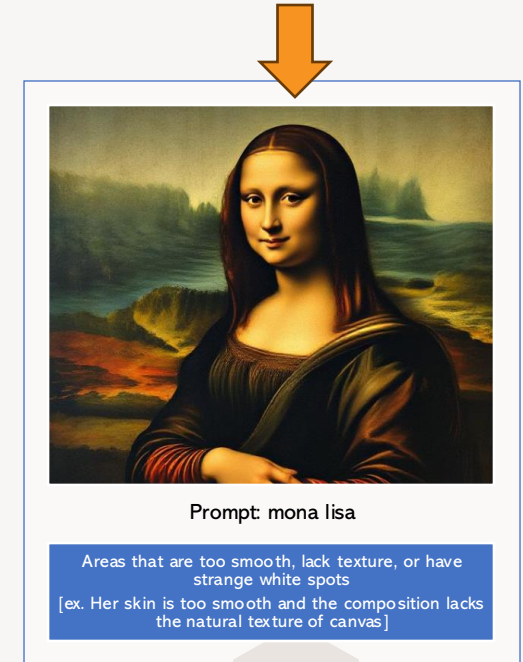
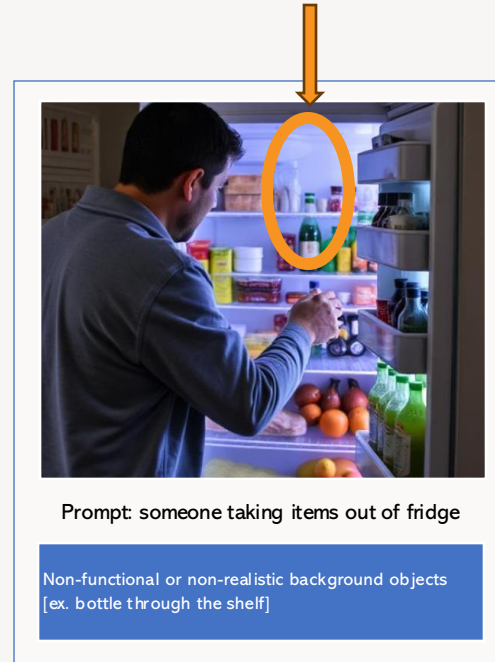
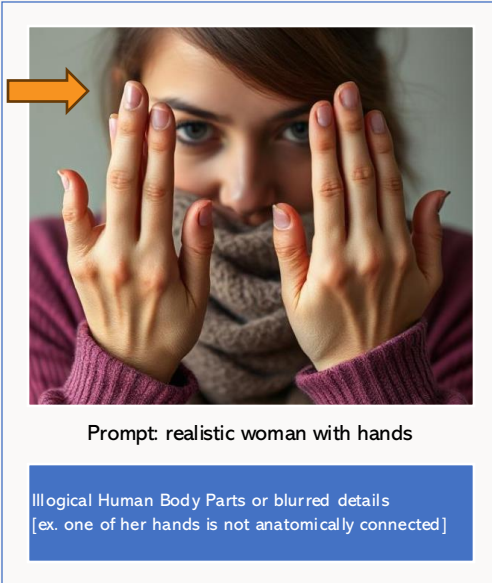
Outlier images where there's not a lot of that per art-type category (i.e. person, real)



Some images are much larger while some are much smaller

How to spot AI?

What types of factors would a human manually try to spot in an image?



- ❖ <https://www.mcgill.ca/oss/article/critical-thinking-technology/how-spot-ai-fakes-now>
- ❖ Images generated using deepai.org (<https://deepai.org/machine-learning-model/text2img>)

Technologies Used

Computing Platforms

Languages



Brief Note on Tools



Machines

- Google Collab (main)
- NVIDIA RTX 3070ti (Emily)



Languages

- Python (main)



Environment

- Whatever Google Collab uses (GPU)
- Python3.9 (Emily – for tensorflow GPU, which required some older versions of certain packages)

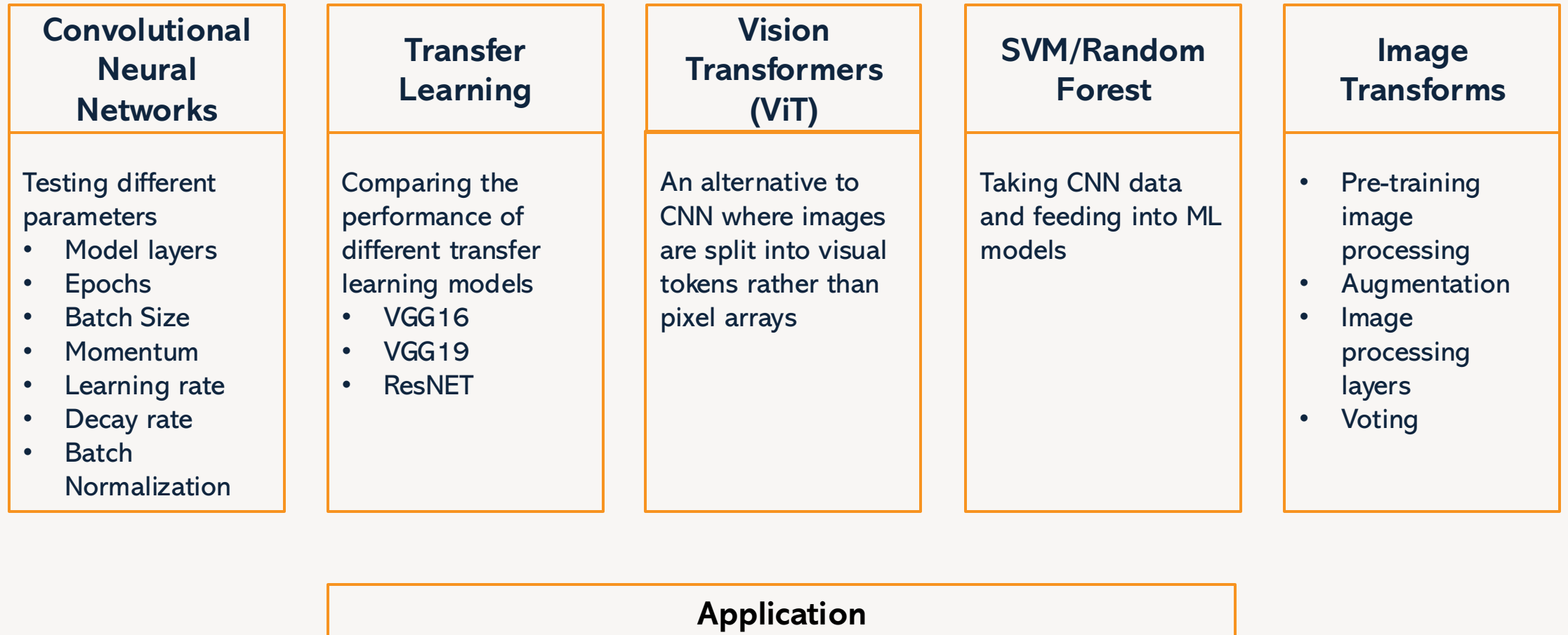
Methodologies

Methodologies Overview

Image Preprocessing Considerations



Methodology Overview



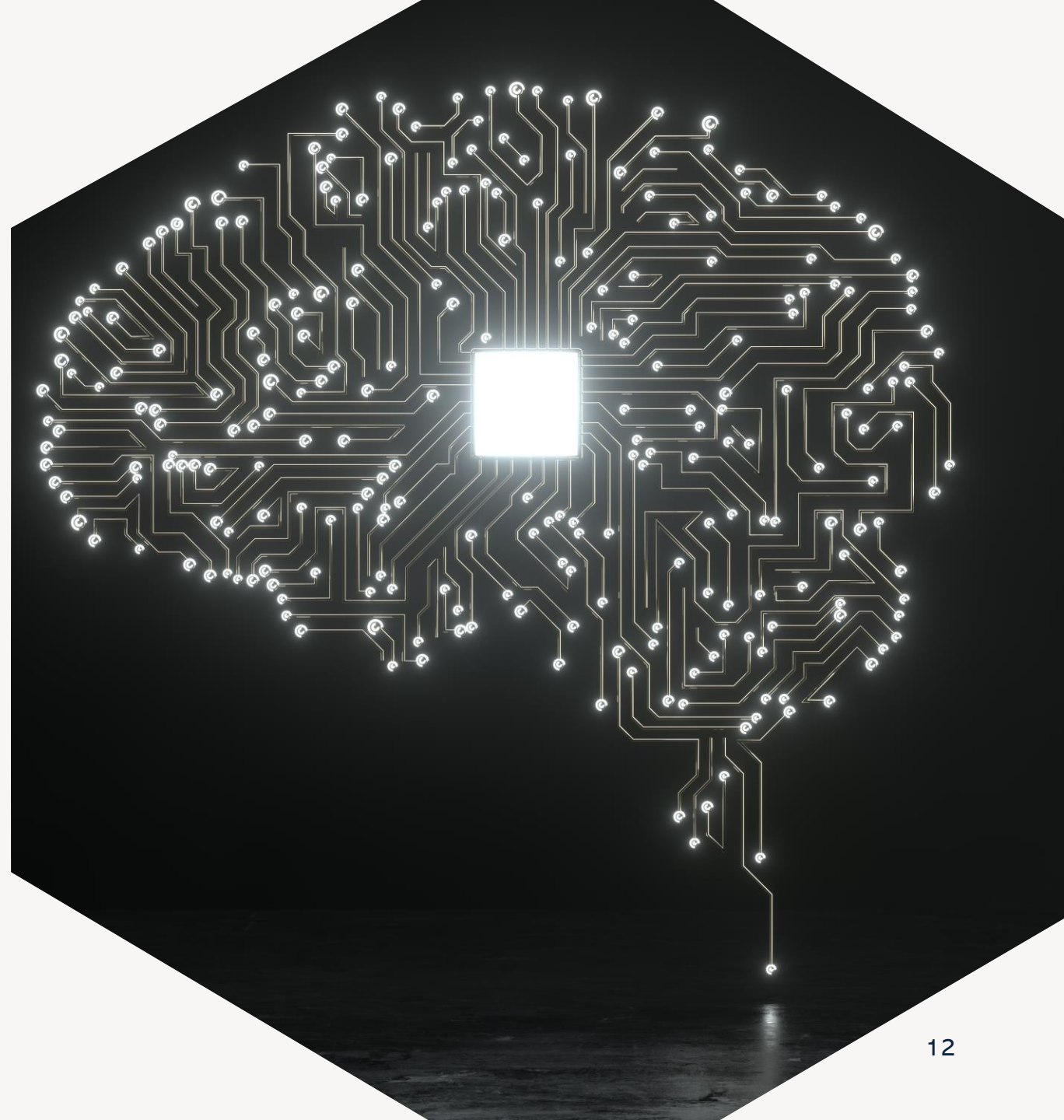
Training and Results

Training

Testing

Metrics

Summaries



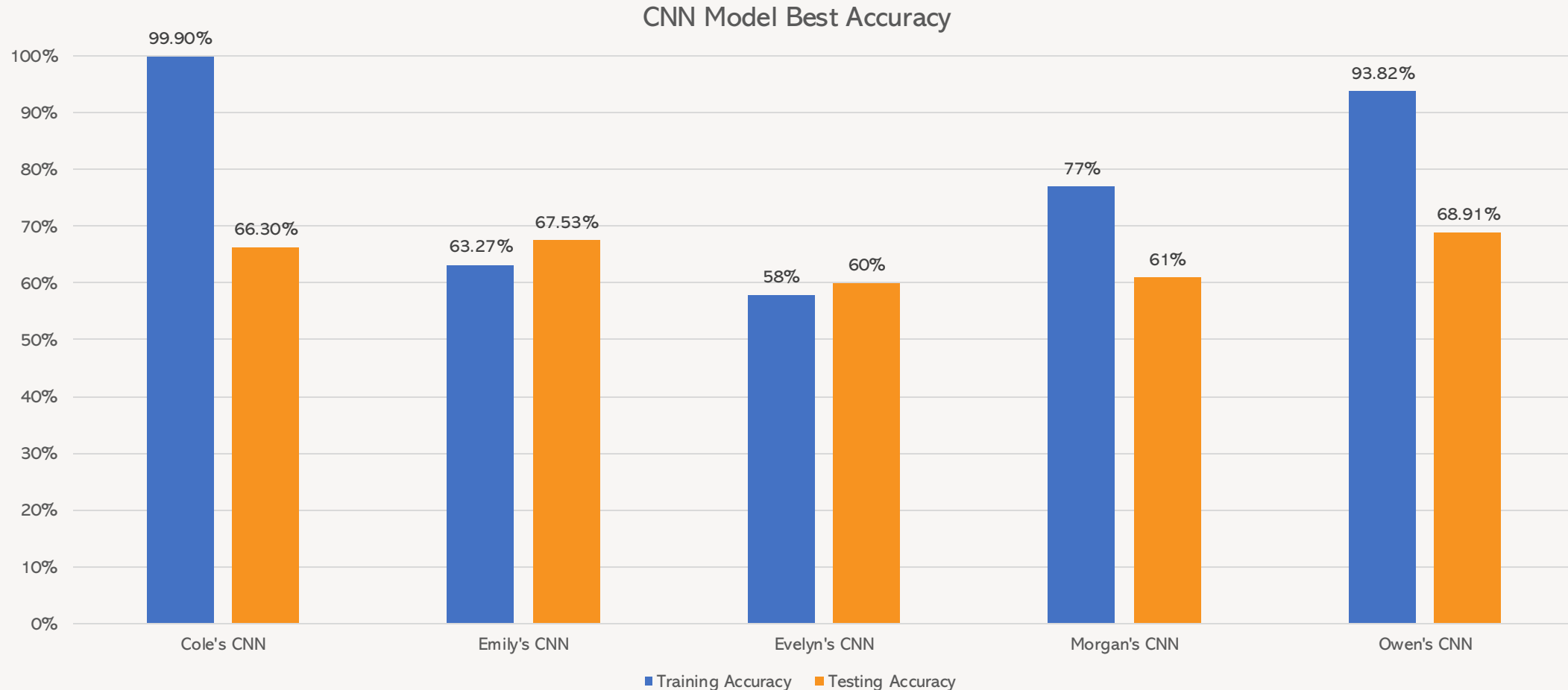


**FROM SCRATCH
EDITING &
TESTING OF
DIFFERENT
PARAMETERS**

Basic Convolutional Neural Networks

From-Scratch Best Models

- Individually everyone tried different combinations of epochs, batch size, learning rate, decay rate, momentum, and model layers → this is a summary of our best results



A large satellite dish antenna is shown from a low angle, looking up. The dish is covered in a grid of small, square panels. A white hexagonal shape is overlaid on the center of the dish, containing the text "DIFFERENT MODELS & TECHNIQUES".

**DIFFERENT
MODELS &
TECHNIQUES**

Transfer Learning

VGG16 : Z-Code

- Feature Maps
 - block3_conv3,
 - block4_conv3,
 - block5_conv3
- T-NSE and UMAP
 - conv layers
 - FC layers

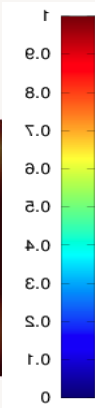
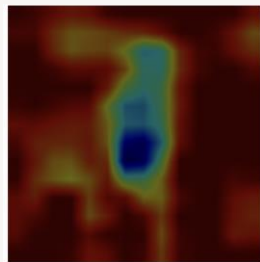
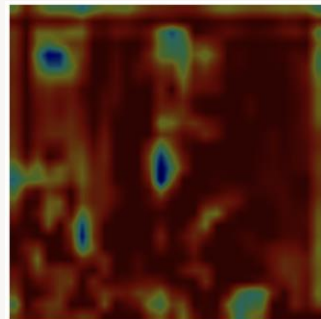
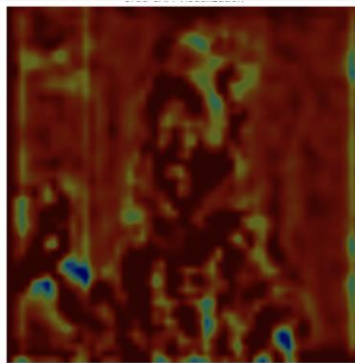
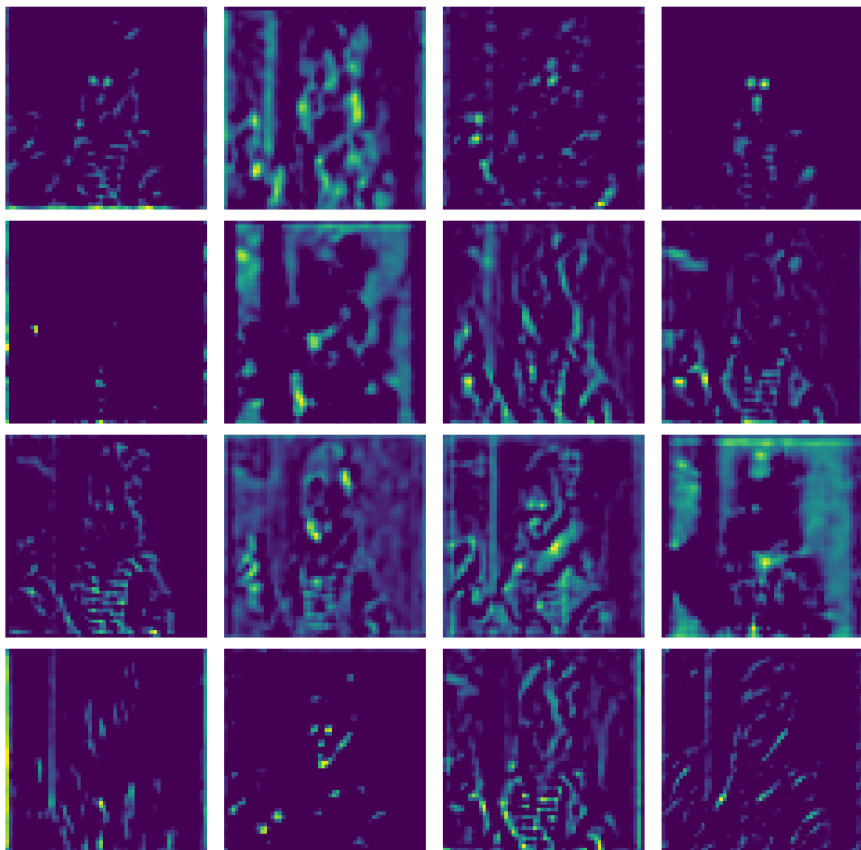
Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 512)	0
dense_5 (Dense)	(None, 256)	131,328
leaky_re_lu_5 (LeakyReLU)	(None, 256)	0
dropout_5 (Dropout)	(None, 256)	0
dense_6 (Dense)	(None, 128)	32,896
leaky_re_lu_6 (LeakyReLU)	(None, 128)	0
dropout_6 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8,256
leaky_re_lu_7 (LeakyReLU)	(None, 64)	0
dropout_7 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 32)	2,080
leaky_re_lu_8 (LeakyReLU)	(None, 32)	0
dropout_8 (Dropout)	(None, 32)	0
dense_9 (Dense)	(None, 8)	264
leaky_re_lu_9 (LeakyReLU)	(None, 8)	0
dropout_9 (Dropout)	(None, 8)	0
dense_output (Dense)	(None, 2)	18

Total params: 14,889,530 (56.80 MB)
 Trainable params: 174,842 (682.98 KB)
 Non-trainable params: 14,714,688 (56.13 MB)

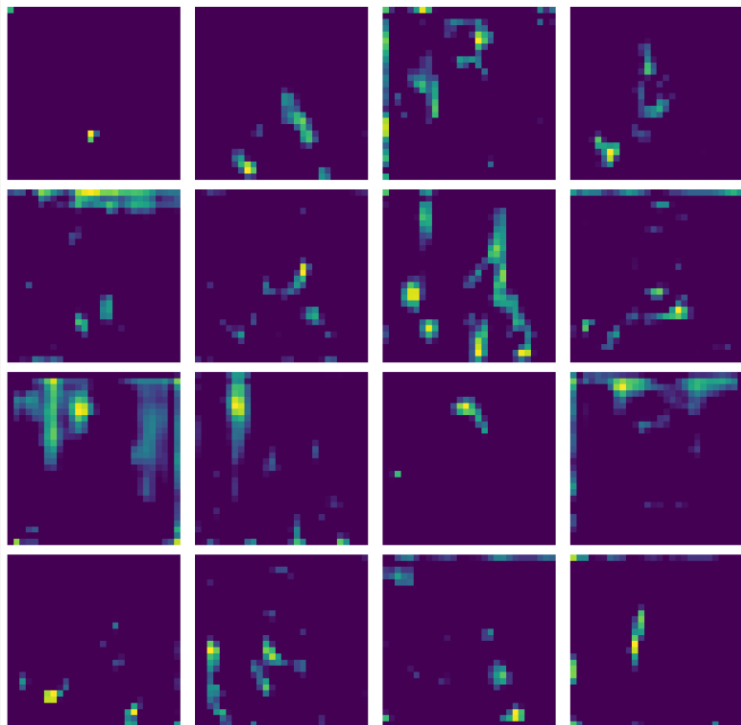
VGG16: Feature Maps

Real Art

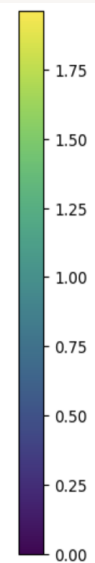
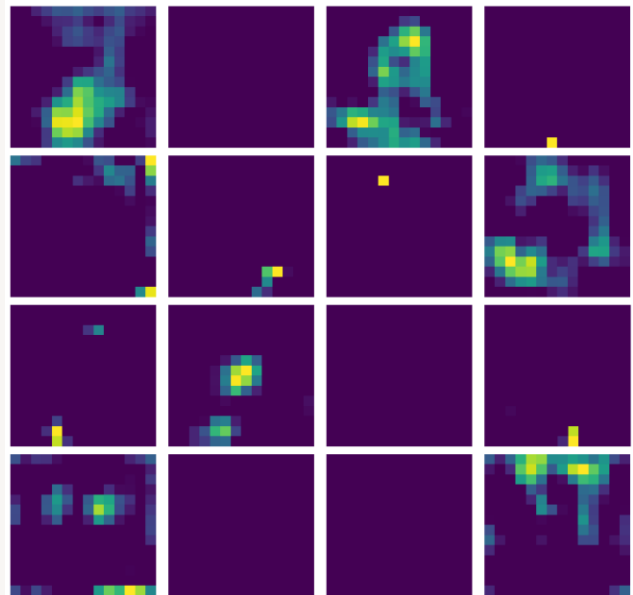
Block3_conv3



Block4_conv3



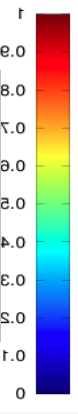
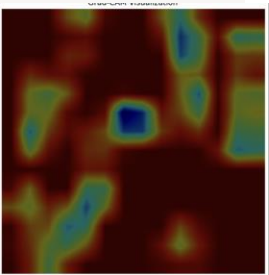
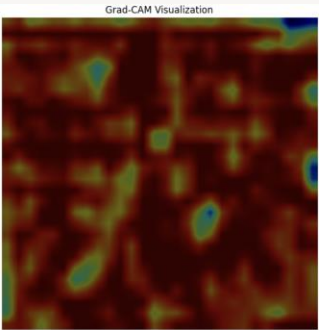
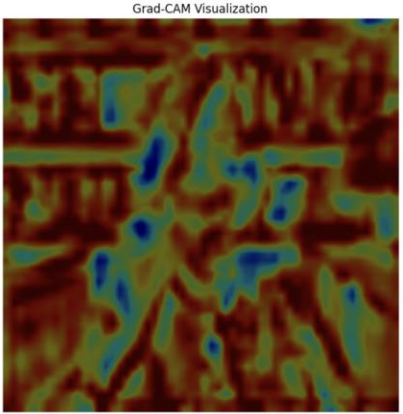
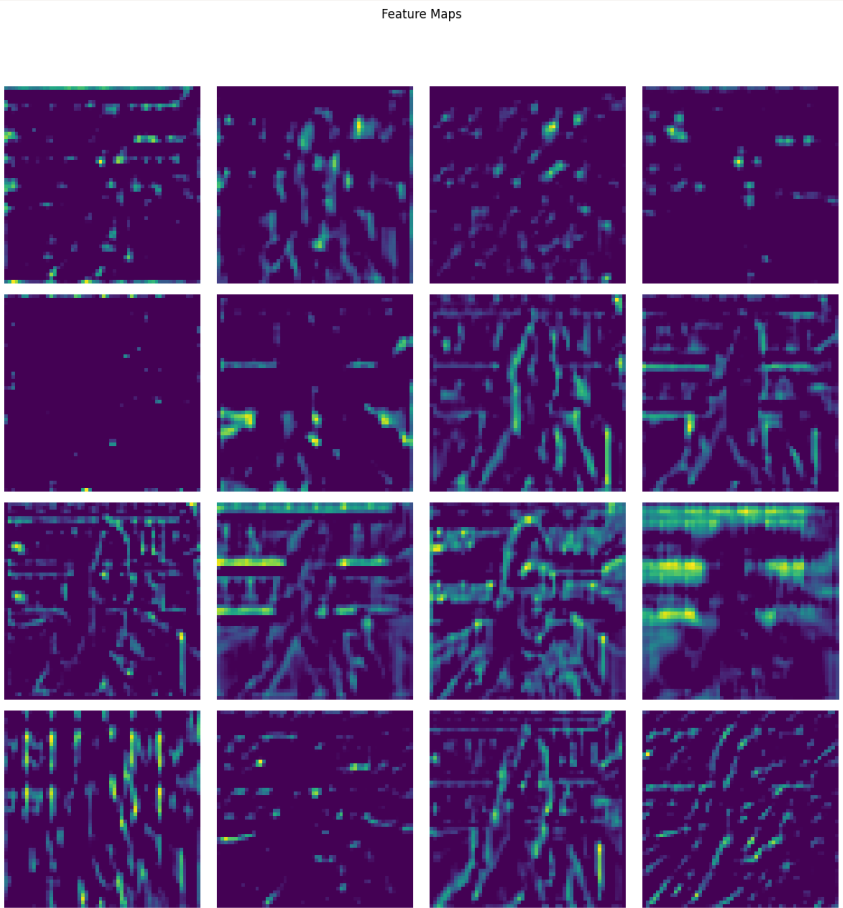
Block5_conv3



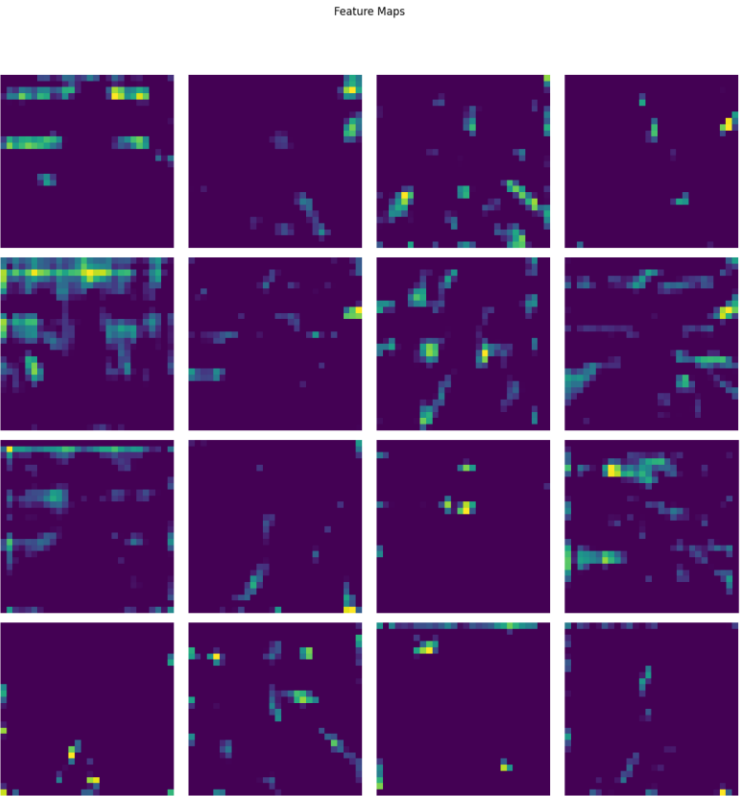
VGG16: Feature Maps

AI Art

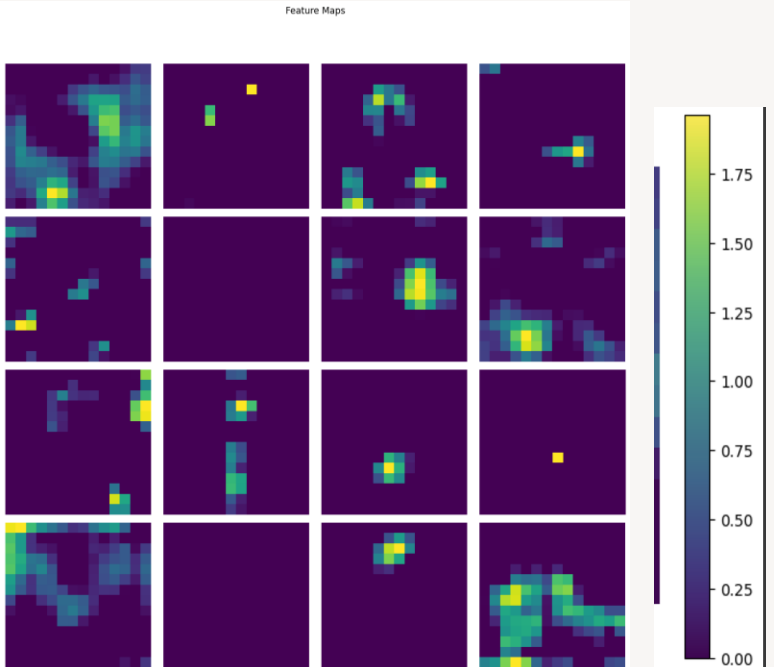
Block3_conv3

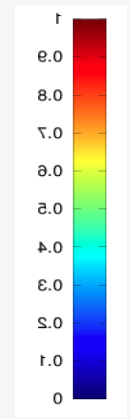


Block4_conv3

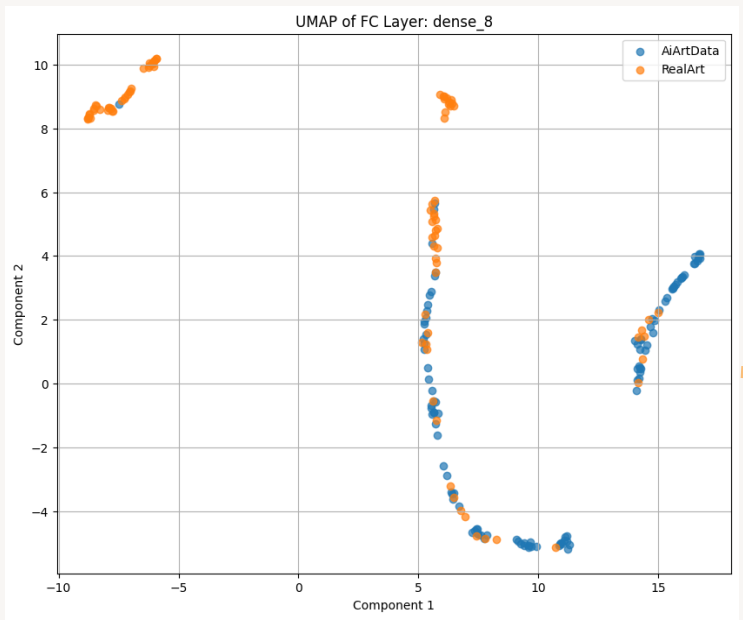
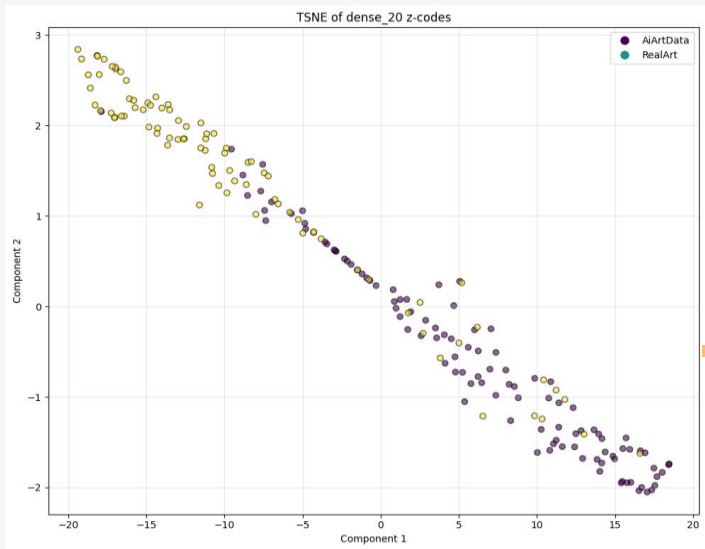
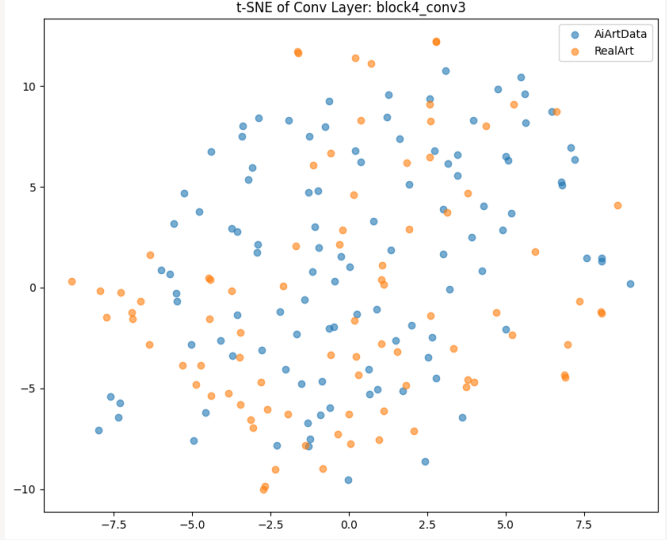
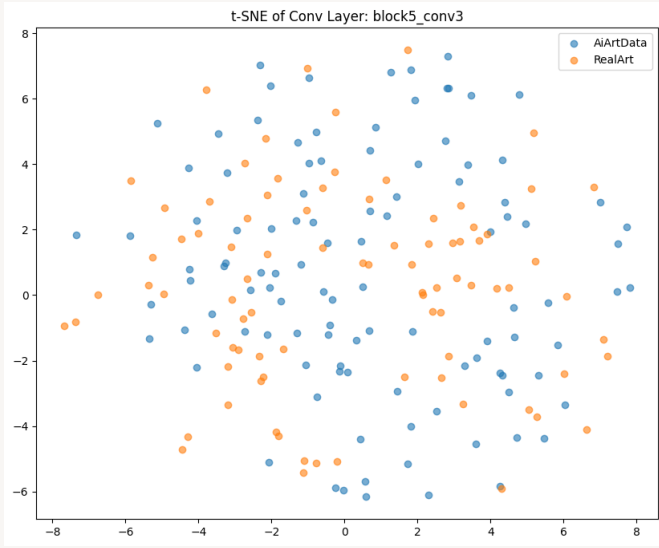


Block5_conv3





VGG16: t-SNE



Layer (type)	Output Shape	Param #
input_layer_1 (InputLayer)	(None, 224, 224, 3)	0
block1_conv1 (Conv2D)	(None, 224, 224, 64)	1,792
block1_conv2 (Conv2D)	(None, 224, 224, 64)	36,928
block1_pool (MaxPooling2D)	(None, 112, 112, 64)	0
block2_conv1 (Conv2D)	(None, 112, 112, 128)	73,856
block2_conv2 (Conv2D)	(None, 112, 112, 128)	147,584
block2_pool (MaxPooling2D)	(None, 56, 56, 128)	0
block3_conv1 (Conv2D)	(None, 56, 56, 256)	295,168
block3_conv2 (Conv2D)	(None, 56, 56, 256)	590,080
block3_conv3 (Conv2D)	(None, 56, 56, 256)	590,080
block3_pool (MaxPooling2D)	(None, 28, 28, 256)	0
block4_conv1 (Conv2D)	(None, 28, 28, 512)	1,180,160
block4_conv2 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_conv3 (Conv2D)	(None, 28, 28, 512)	2,359,808
block4_pool (MaxPooling2D)	(None, 14, 14, 512)	0
block5_conv1 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv2 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_conv3 (Conv2D)	(None, 14, 14, 512)	2,359,808
block5_pool (MaxPooling2D)	(None, 7, 7, 512)	0
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 512)	0
dense_5 (Dense)	(None, 256)	131,328
leaky_re_lu_5 (LeakyReLU)	(None, 256)	0
dropout_5 (Dropout)	(None, 256)	0
dense_6 (Dense)	(None, 128)	32,896
leaky_re_lu_6 (LeakyReLU)	(None, 128)	0
dropout_6 (Dropout)	(None, 128)	0
dense_7 (Dense)	(None, 64)	8,256
leaky_re_lu_7 (LeakyReLU)	(None, 64)	0
dropout_7 (Dropout)	(None, 64)	0
dense_8 (Dense)	(None, 32)	2,080
leaky_re_lu_8 (LeakyReLU)	(None, 32)	0
dropout_8 (Dropout)	(None, 32)	0
dense_9 (Dense)	(None, 8)	264
leaky_re_lu_9 (LeakyReLU)	(None, 8)	0
dropout_9 (Dropout)	(None, 8)	0
dense_output (Dense)	(None, 2)	18

Total params: 14,889,530 (56.80 MB)
Trainable params: 174,842 (682.98 KB)
Non-trainable params: 14,714,688 (56.13 MB)

VGG19

Architecture:

Layer (type)	Output Shape	Param #
vgg19 (Functional)	(None, 7, 7, 512)	20,024,384
flatten (Flatten)	(None, 25088)	0
dense (Dense)	(None, 512)	12,845,568
dropout (Dropout)	(None, 32)	0
dense_1 (Dense)	(None, 256)	131,328
dropout (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 128)	32,896
dropout (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 64)	8,256
dropout (Dropout)	(None, 32)	0
dense_4 (Dense)	(None, 32)	2,080
dropout (Dropout)	(None, 32)	0
dense_5 (Dense)	(None, 16)	528
dense_6 (Dense)	(None, 2)	34

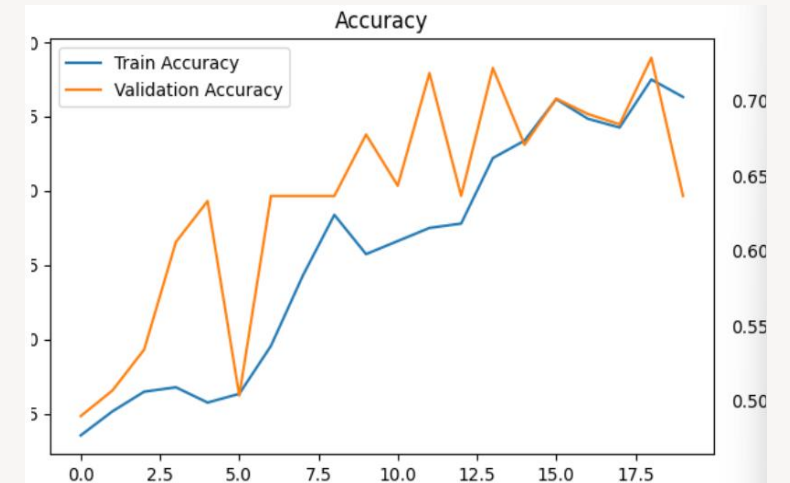
- VGG19 confusion matrix:

```

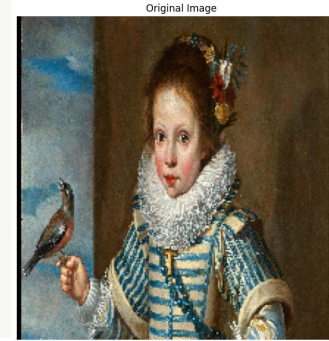
Accuracy: 0.748
Confusion Matrix:
[[126  34]
 [ 39  91]]
Classification Report:
              precision    recall  f1-score   support

     0       0.76       0.79       0.78       160
     1       0.73       0.70       0.71       130

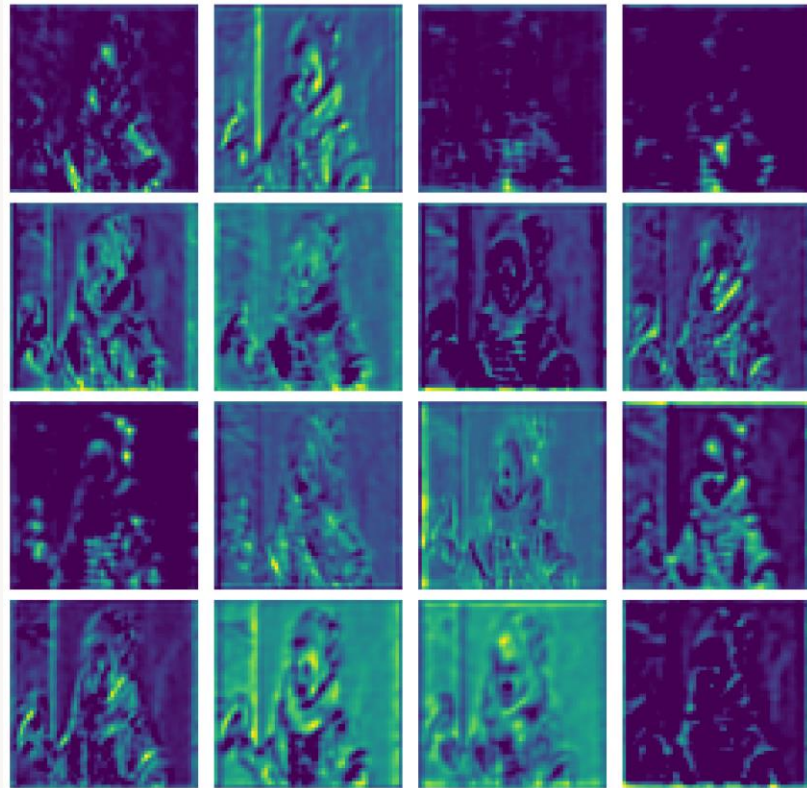
   accuracy          0.75
  macro avg       0.75       0.74       0.74       290
 weighted avg       0.75       0.75       0.75       290
    
```



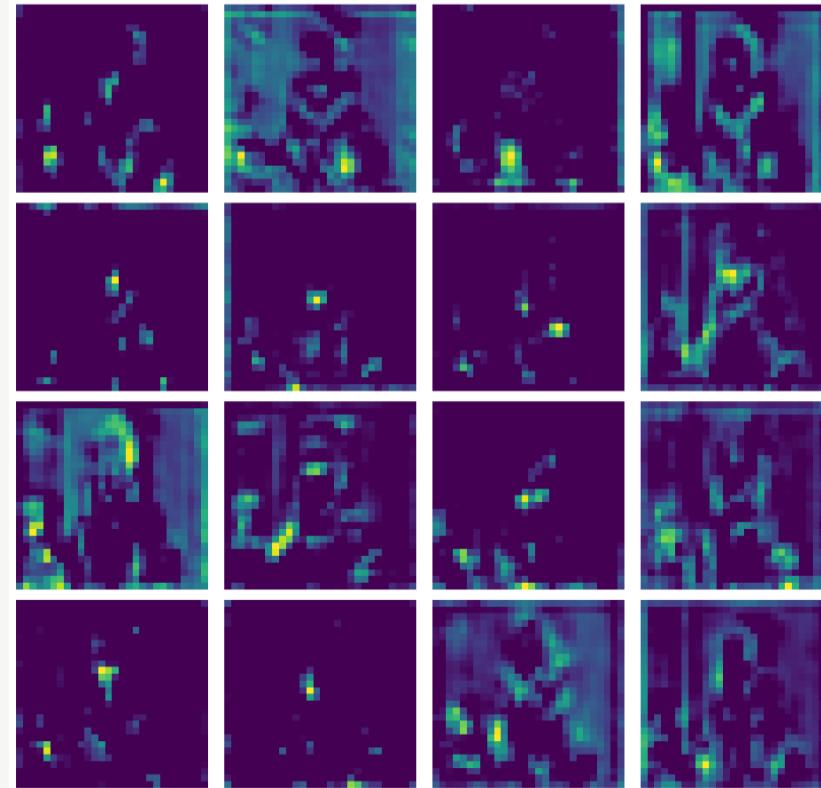
Feature Maps- VGG19



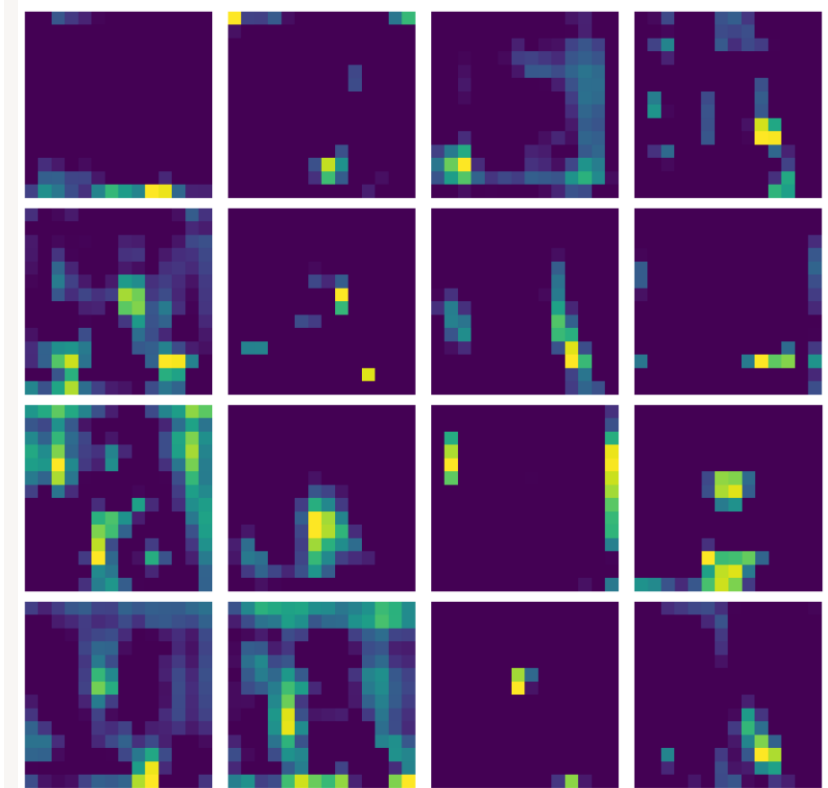
Mid-Level



High-Level

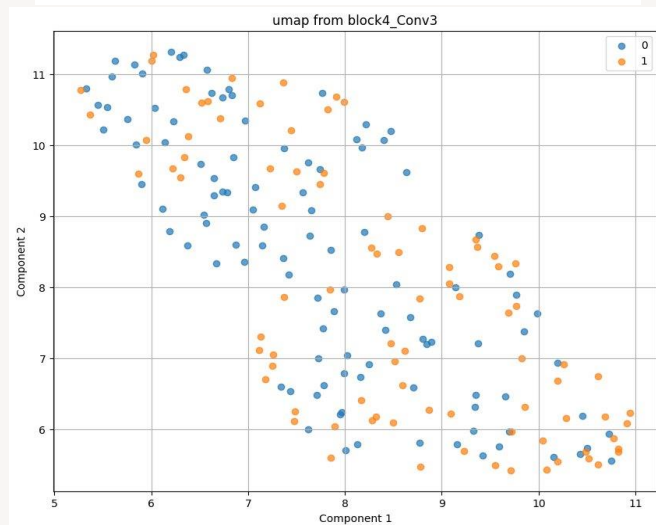
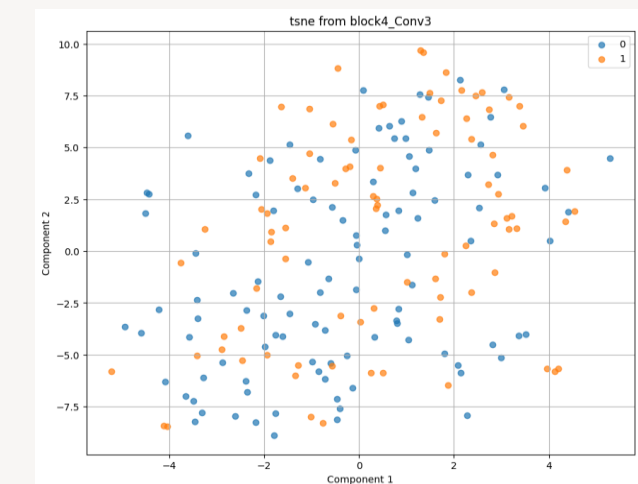


Highest-Level

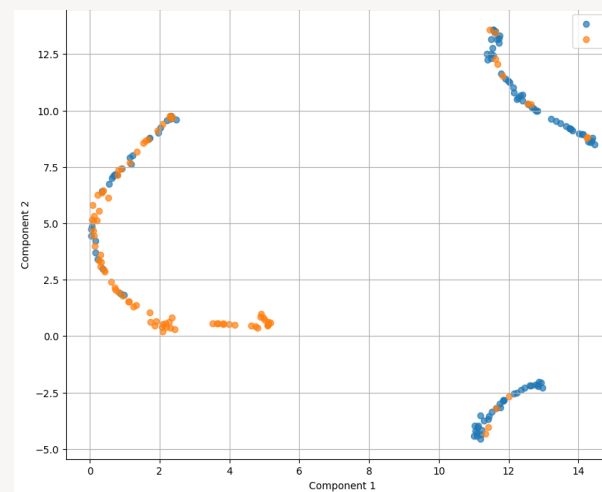
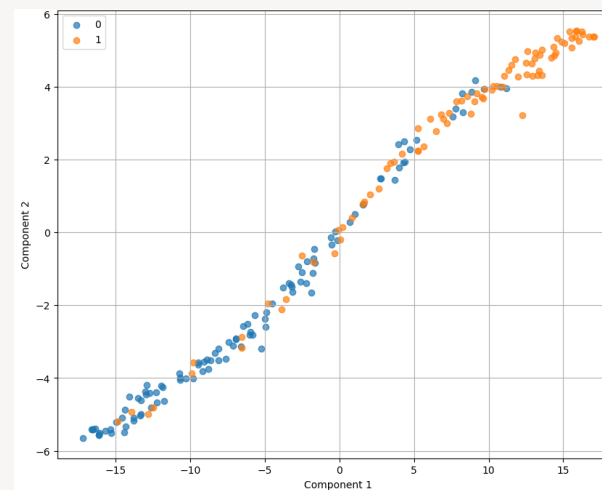


Z-code & t-sne VGG19

Block4_conv3



Dense_5



ResNet50

Architecture

Layer (type)	Output Shape	Param #
resnet50 (Functional)	(None, 7, 7, 2048)	23,587,712
flatten (Flatten)	(None, 100352)	0
dense (Dense)	(None, 2048)	205,522,944
dense_1 (Dense)	(None, 512)	1,049,088
dense_2 (Dense)	(None, 128)	65,664
dense_3 (Dense)	(None, 32)	4,128
dense_4 (Dense)	(None, 8)	264
dense_5 (Dense)	(None, 2)	18

Best Hyperparameters

Size: 224x224

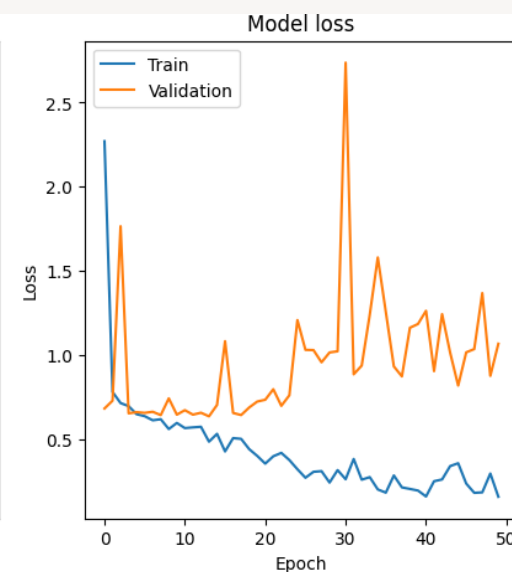
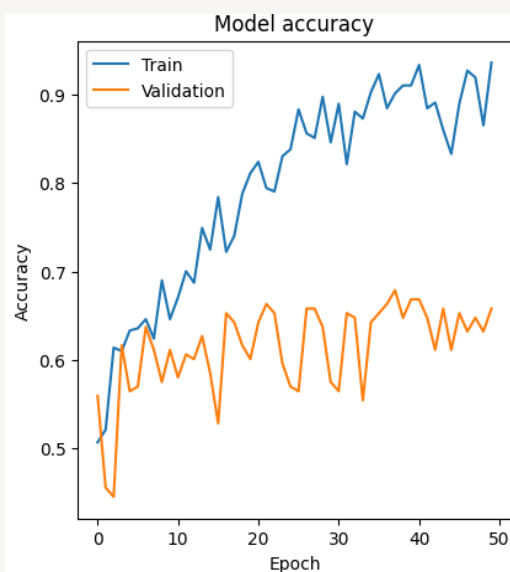
Frozen (no retraining)

Epochs: 50

Batch Size: 10

Best Run Accuracy:

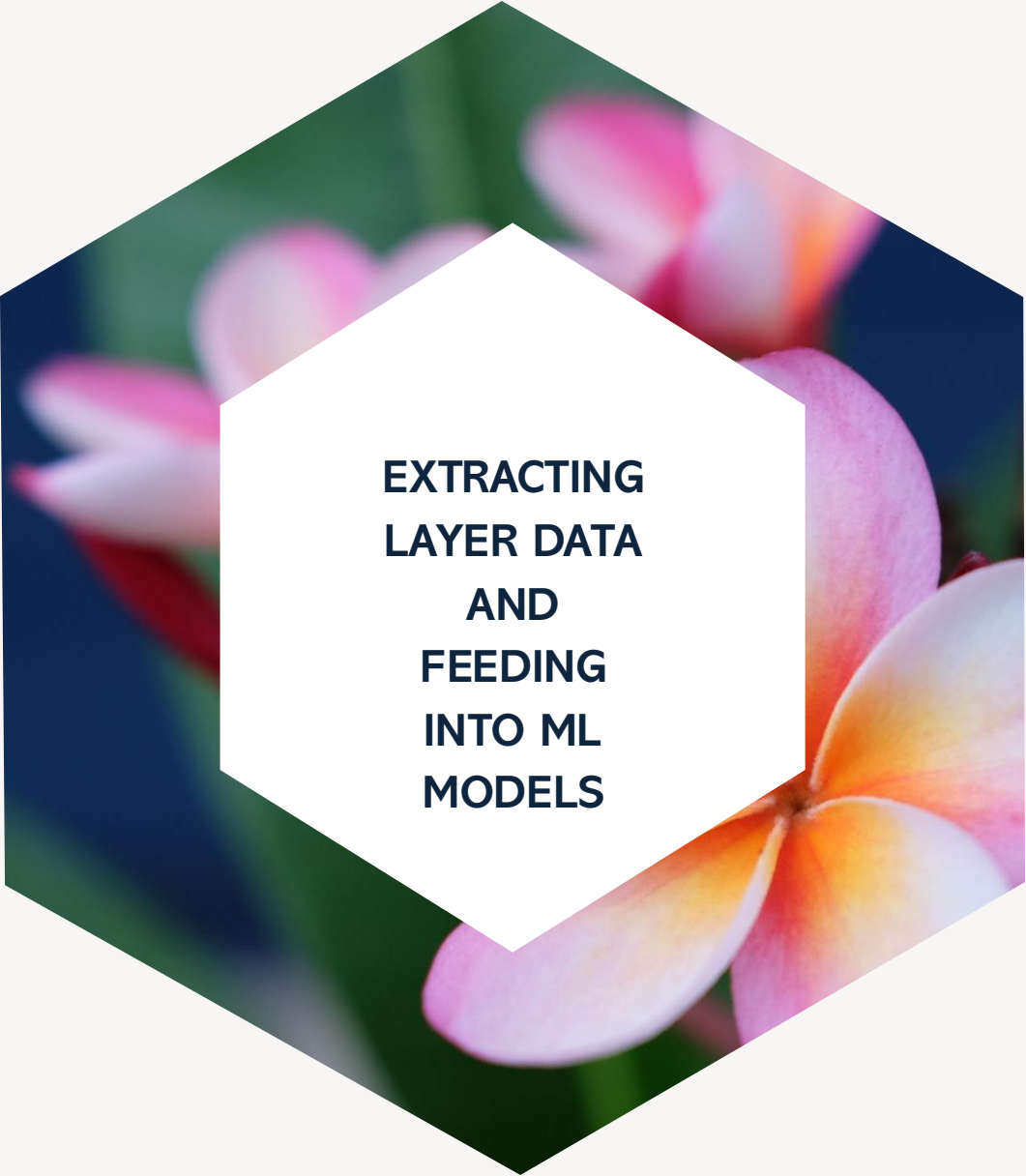
- Training: 92.2%
- Test: 67.9%
- Val Loss: 0.87



Accuracy: 0.658

CFM:
[[82 25]
[41 45]]

Classification report:		precision	recall	f1-score	support
	0	0.67	0.77	0.71	107
	1	0.64	0.52	0.58	86
accuracy				0.66	193
macro avg		0.65	0.64	0.64	193
weighted avg		0.66	0.66	0.65	193

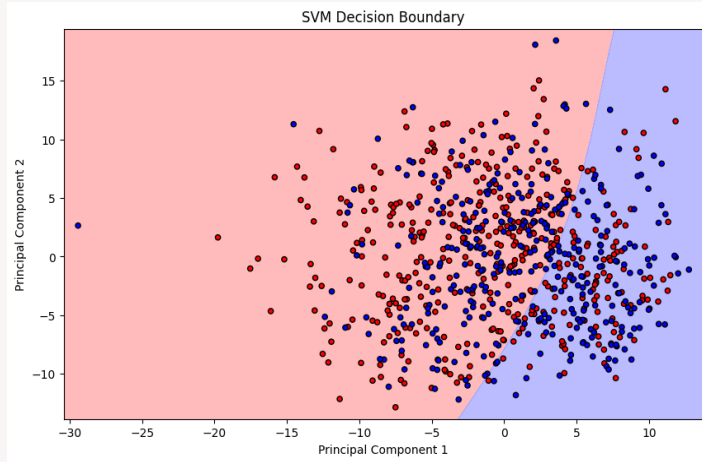


**EXTRACTING
LAYER DATA
AND
FEEDING
INTO ML
MODELS**

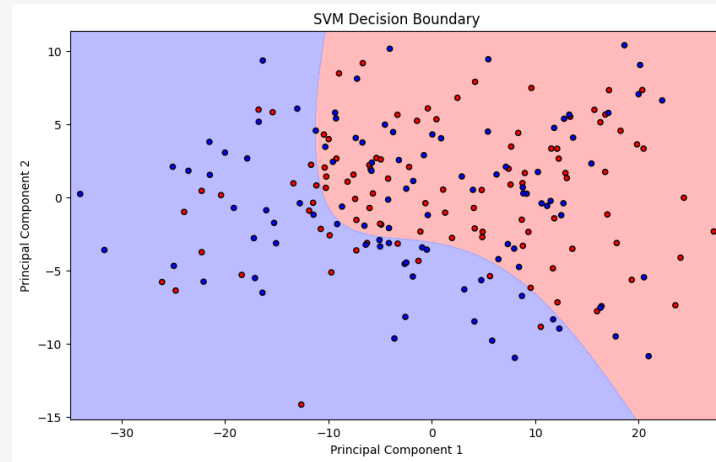
CNN with ML models

Z- code with SVM: VGG16

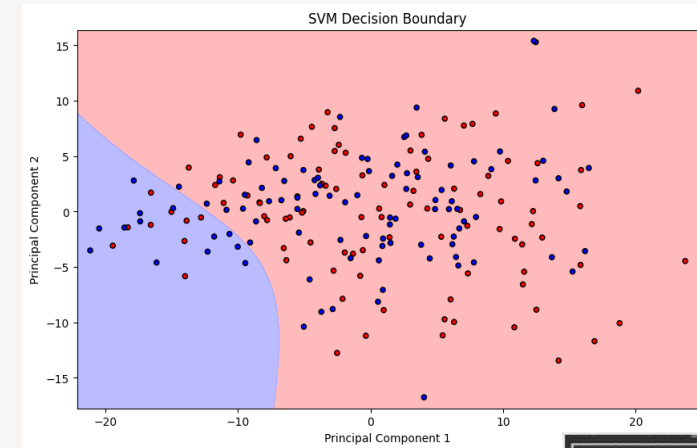
Block5_Conv3: 81.87%



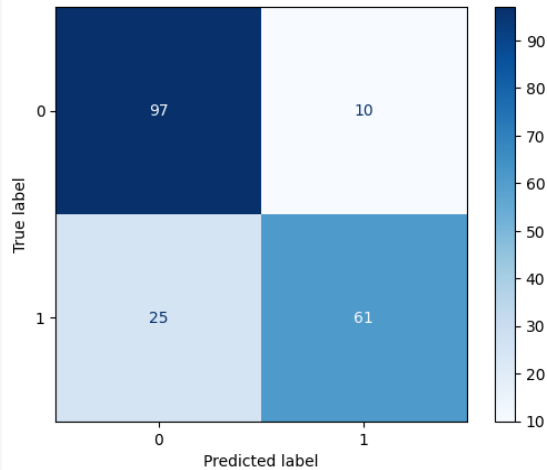
Block4_Conv3



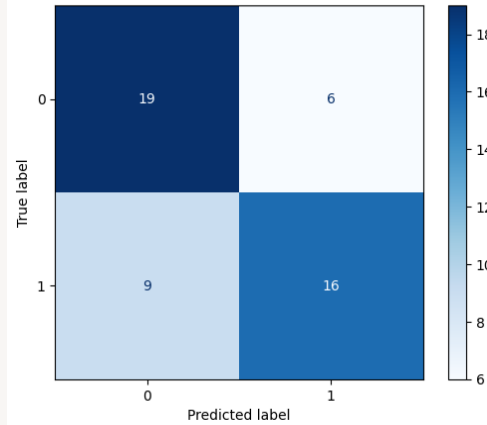
Block3_Conv3



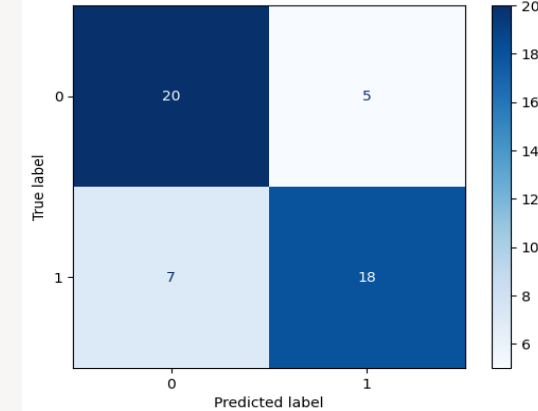
SVM Confusion Matrix



SVM Confusion Matrix



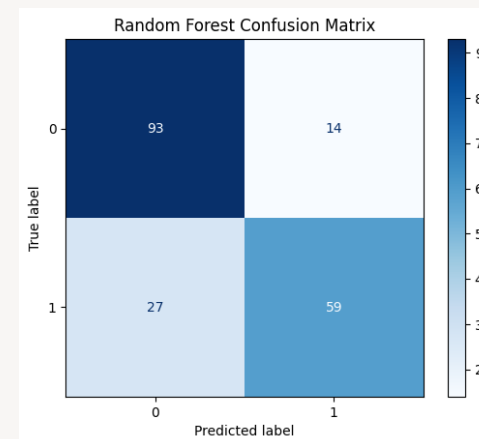
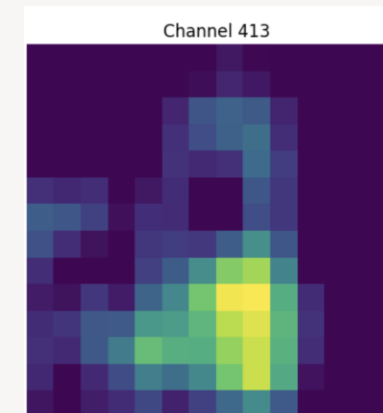
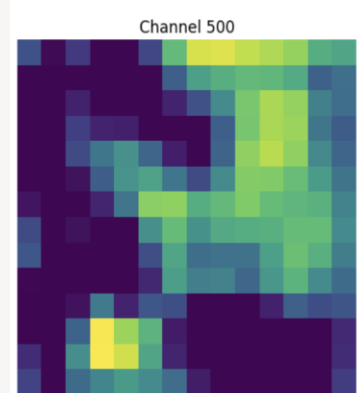
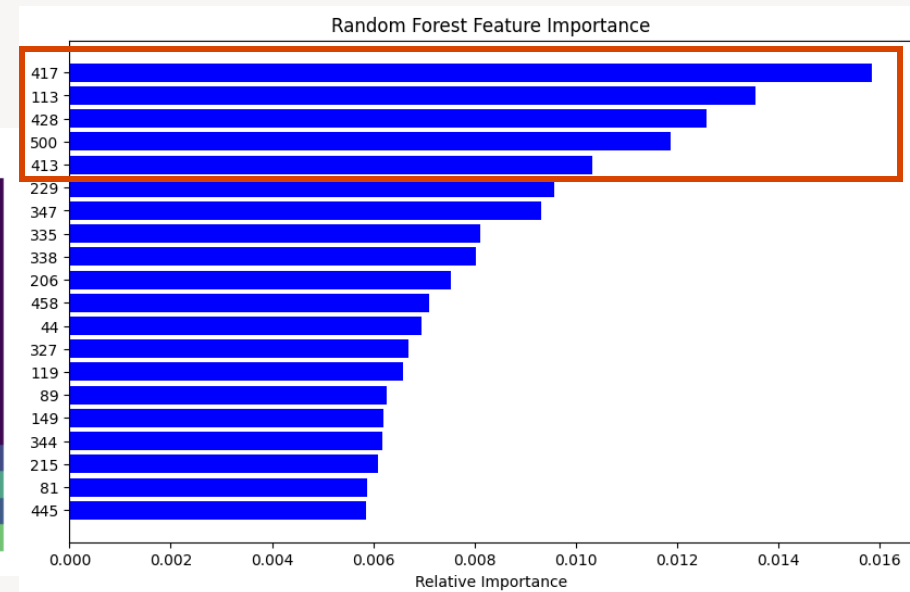
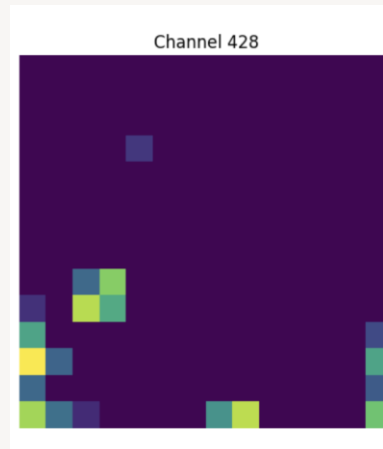
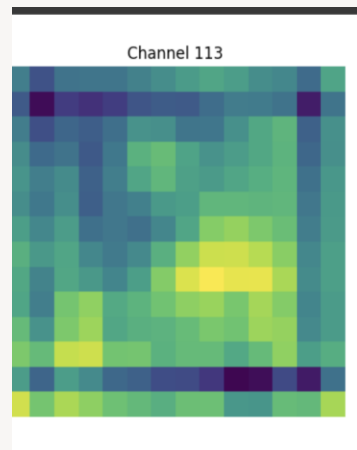
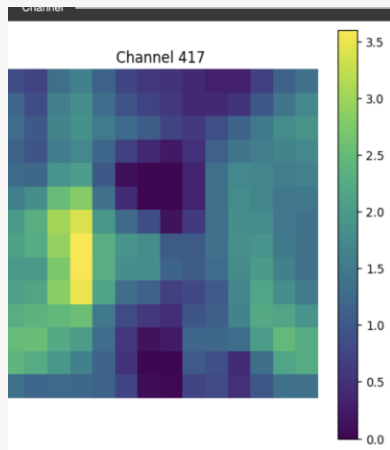
SVM Confusion Matrix



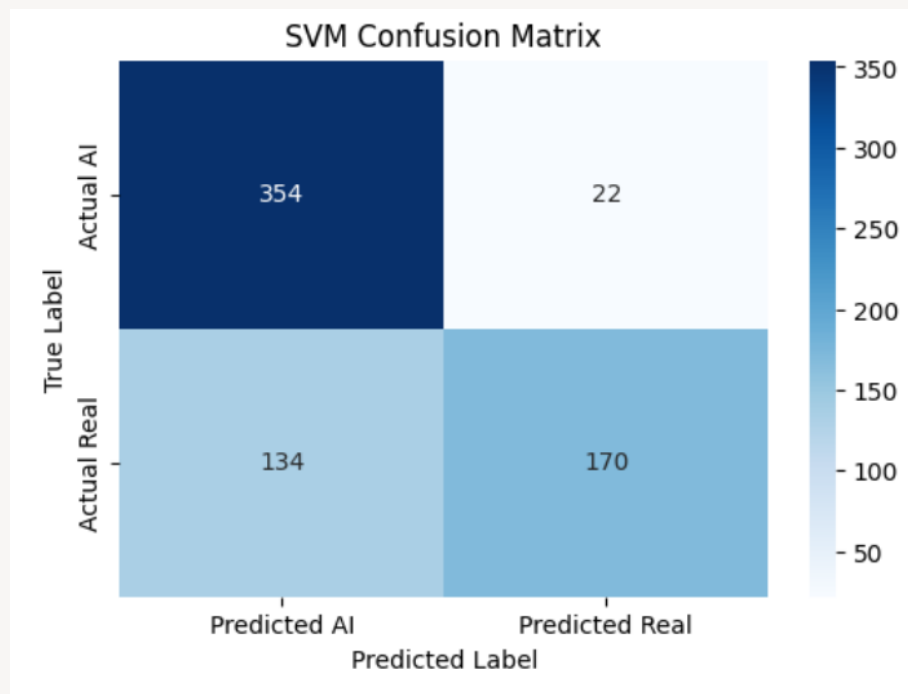
Layer (type)	Output Shape
input_layer_1 (InputLayer)	(None, 224, 224, 3)
block1_conv1 (Conv2D)	(None, 224, 224, 64)
block1_conv2 (Conv2D)	(None, 224, 224, 64)
block1_pool (MaxPooling2D)	(None, 112, 112, 64)
block2_conv1 (Conv2D)	(None, 112, 112, 128)
block2_conv2 (Conv2D)	(None, 112, 112, 128)
block2_pool (MaxPooling2D)	(None, 56, 56, 128)
block3_conv1 (Conv2D)	(None, 56, 56, 256)
block3_conv2 (Conv2D)	(None, 56, 56, 256)
block3_conv3 (Conv2D)	(None, 56, 56, 256)
block3_pool (MaxPooling2D)	(None, 28, 28, 256)
block4_conv1 (Conv2D)	(None, 28, 28, 512)
block4_conv2 (Conv2D)	(None, 28, 28, 512)
block4_conv3 (Conv2D)	(None, 28, 28, 512)
block4_pool (MaxPooling2D)	(None, 14, 14, 512)
block5_conv1 (Conv2D)	(None, 14, 14, 512)
block5_conv2 (Conv2D)	(None, 14, 14, 512)
block5_conv3 (Conv2D)	(None, 14, 14, 512)
block5_pool (MaxPooling2D)	(None, 7, 7, 512)
global_average_pooling2d_1 (GlobalAveragePooling2D)	(None, 512)
dense_5 (Dense)	(None, 256)

VGG16: Z-code with Random Forest

ACCURACY: 78.76%
Conv Layer: **Block5_Conv3**

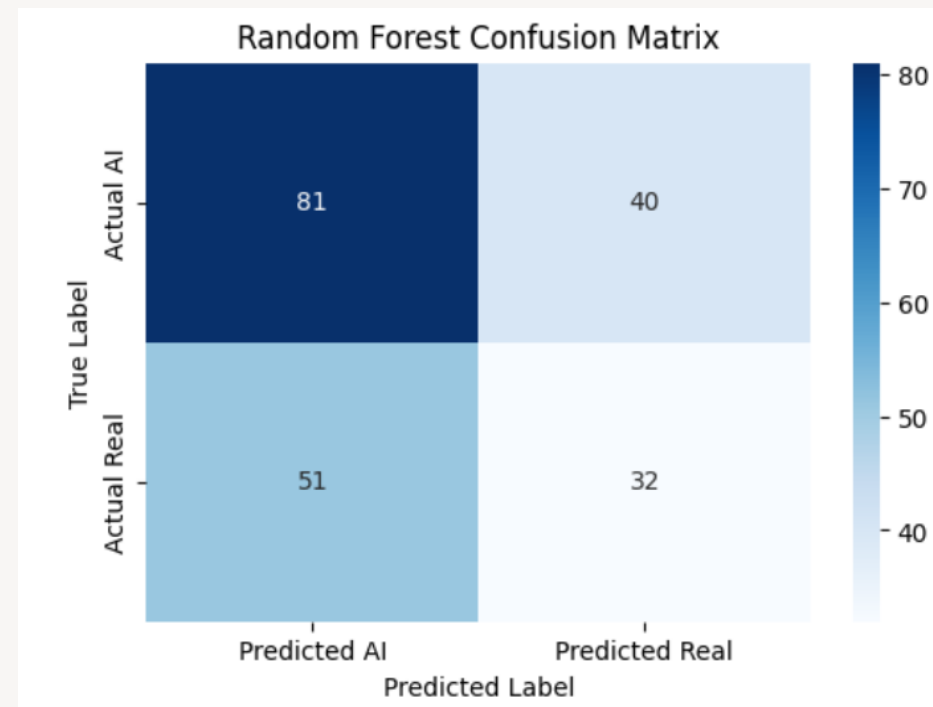


ML Models using Z-code from VGG19



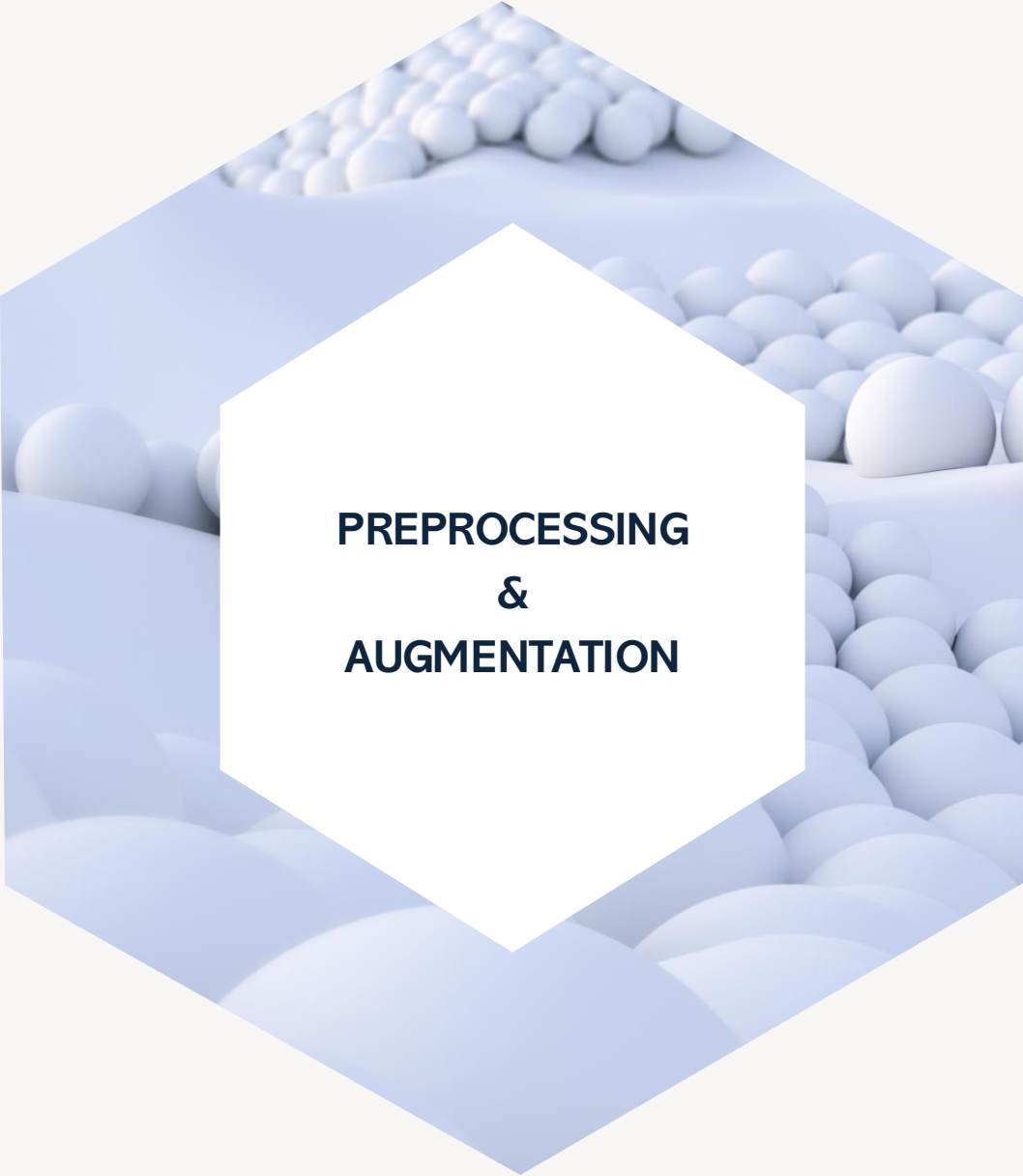
SVM Accuracy: 0.7705882352941177

	precision	recall	f1-score	support
0.0	0.73	0.94	0.82	376
1.0	0.89	0.56	0.69	304
accuracy			0.77	680
macro avg	0.81	0.75	0.75	680
weighted avg	0.80	0.77	0.76	680



Train Accuracy: 1.0
Test Accuracy: 0.553921568627451
Random Forest Accuracy: 1.0

	precision	recall	f1-score	support
AI	0.61	0.67	0.64	121
Real	0.44	0.39	0.41	83
accuracy			0.55	204
macro avg	0.53	0.53	0.53	204



**PREPROCESSING
&
AUGMENTATION**

Image Transforms

Goal – check to see if transforms may improve predictions of “problematic” images

Flipping on an Axis



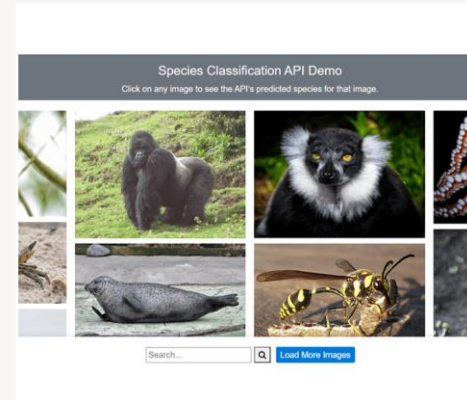
May adjust perspectives of some images and make the training more robust on different orientations?

Rotation



Some images have faces or objects that are slightly tilted, could reorienting help?

Crop / Zoom



Could randomly cropping images improve prediction of composite images?

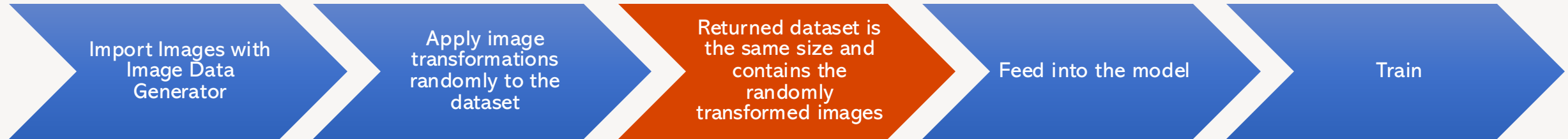
Brightness



Could increasing or decreasing the brightness enhance certain features in an image?

Image Transforms on the training set

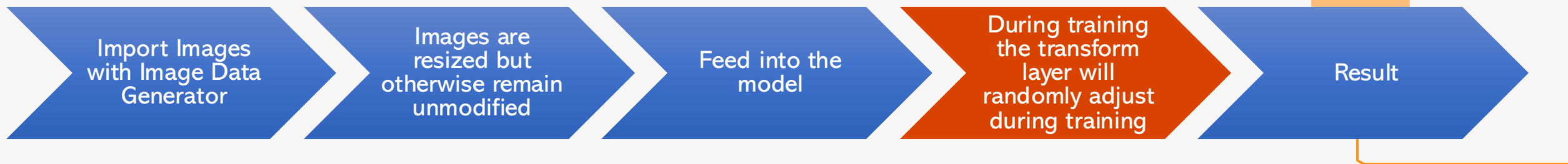
Method #1: Pre-processing – randomly replace images with a transformed version prior to training



Method #2: Augmentation – append transformed images to the training set



Method #3: Model Layers – randomly applies a transform to the images during training



Method #1: Pre-processing

- Image Data Generator to randomly select images to replace prior to training → will return a data set of the same-size

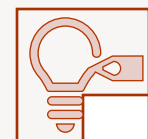
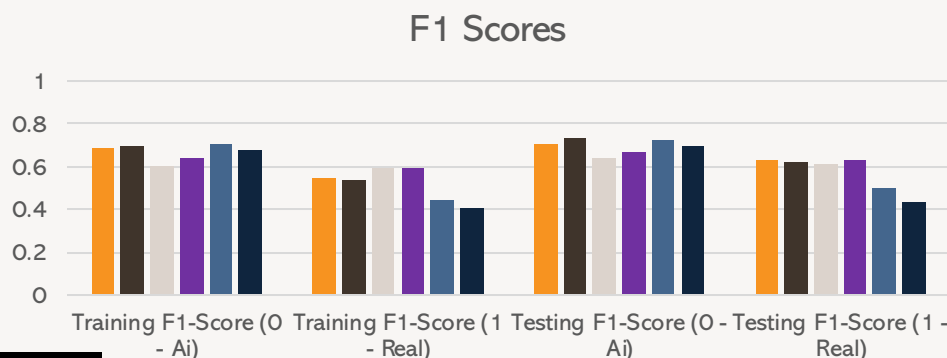
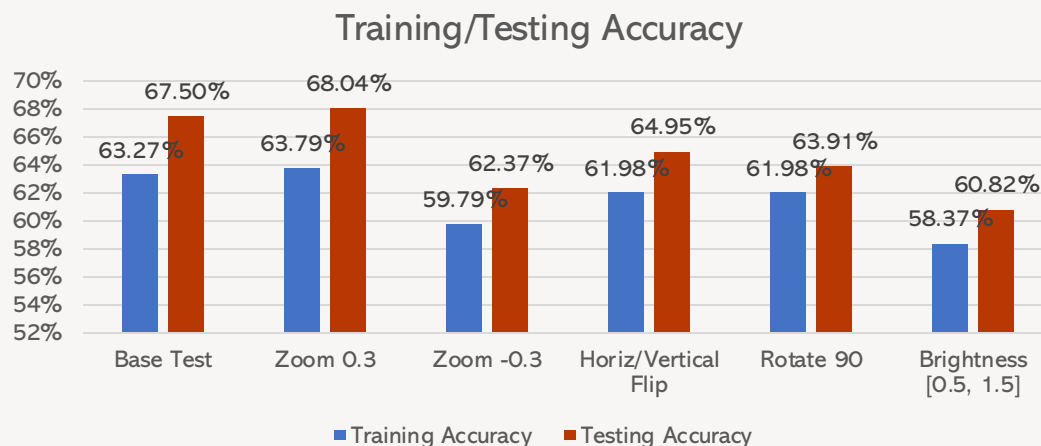


Image Data Generator

1. Take saved train split & run through ImageDataGenerator transforms
2. Returns a same-sized dataset of replaced transformed images
3. Run through training
4. Compare results to Test images (untransformed)

- Training time: 14-18min
- Model layers are the same between tests & only changing the images

```
data = ImageDataGenerator(rescale=1./255,  
validation_split=0.0,  
zoom_range=-0.3  
horizontal_flip=True,  
vertical_flip=True  
rotation_range=90  
brightness_range = [0.5, 1.5]  
)
```



Base Test Zoom 0.3 Zoom -0.3
Horiz/Vertical Flip Rotate 90 Brightness [0.5, 1.5]

Overall, no real improvements to the metrics after doing image-preprocessing

Some techniques like changing the brightness or negative zoom

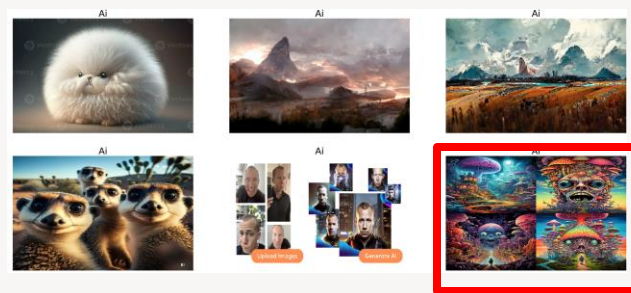
Method #1: Pre-processing

Comparison of the first 6 mispredicted images (images are numerically in order for all tests)

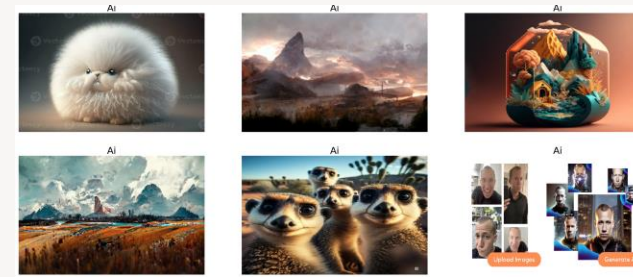
Base Test



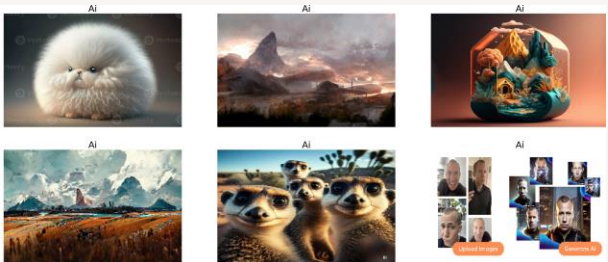
Zoom 0.3



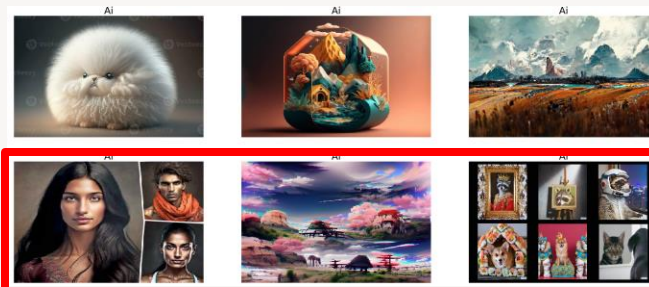
Zoom -0.3



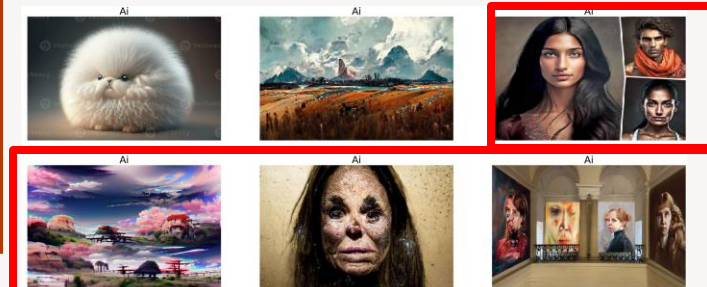
Rand Flip



Rotate 90



Brightness



While the overall metrics remained about the same, the key is that some of the image transforms led to better predictions on certain images → some mispredicted (fairly consistently) in the non-transformed tests were predicted correctly in the transform tests

Method #2: Augmentation

- F1 Scores for Testing ranged from 0.68 – 0.79
- Best results were with the VGG16 rather than from-scratch models when using augmented dataset
- Training for longer didn't improve results by much, on average within the 60-76% range
- Some models exhibited overfitting or stagnation



e Data Generator

Test #1 (Augmentation)

1. Take the saved train split and run through Image Data Generator
2. Save a sample of 30 images to a folder for a new augmented training set
3. Repeat steps 1 & 2 for different transforms
4. Result = new augmented dataset with appended training images

Test #2 (zoom-only)

- Saved different Zoom ranges (50 img each)

Test #3 (Augmentation 2)

- Saved 50 images each for
1. Greyscale
 2. Zoom -0.5
 3. Zoom 0.3
 4. Zoom 0.8
 5. Horizontal flip
 6. Rotate 90

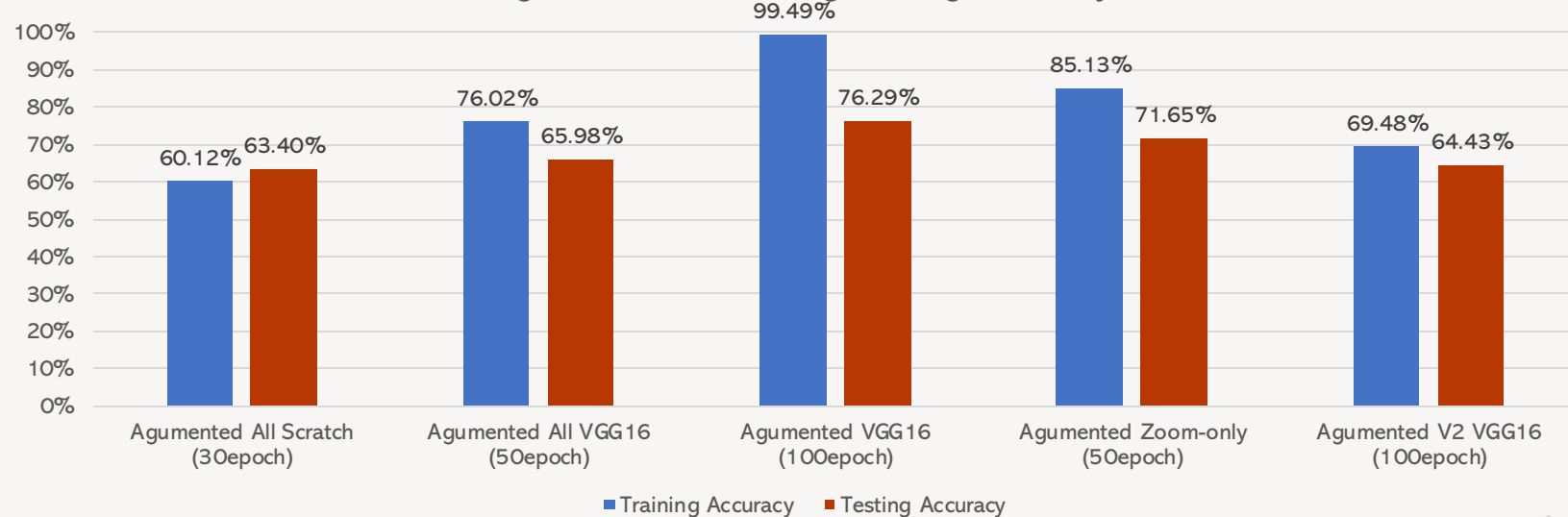
Test #1

zoom_range=-0.5
horizontal_flip=True & vertical_flip=True
rotation_range=90
brightness_range = [0.5, 1.5]

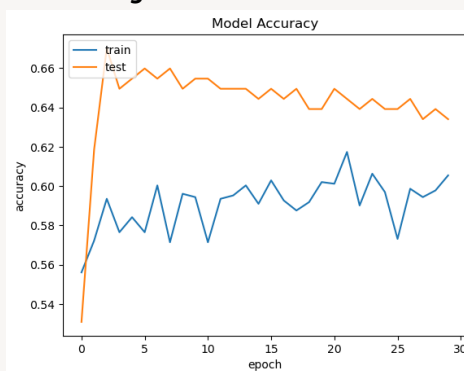
Test #2:

Zoom: 0.3, -0.6, 0.8

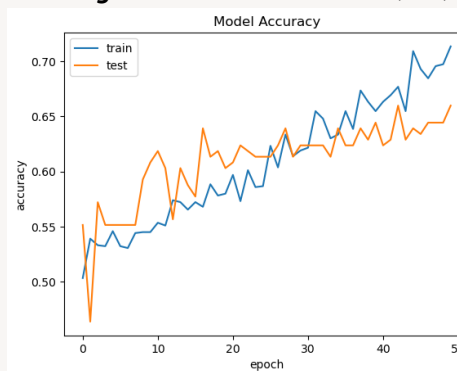
Augmentation Training/Testing Accuracy



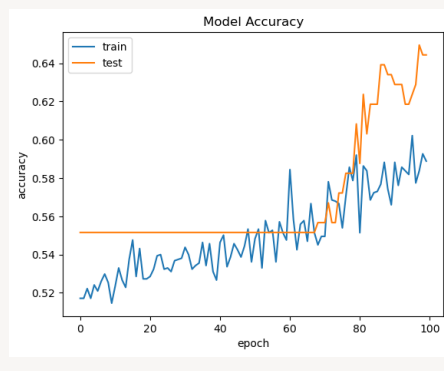
Augmented All Scratch



Augmented All VGG16 (50)



Augmented V2 VGG16 (100)

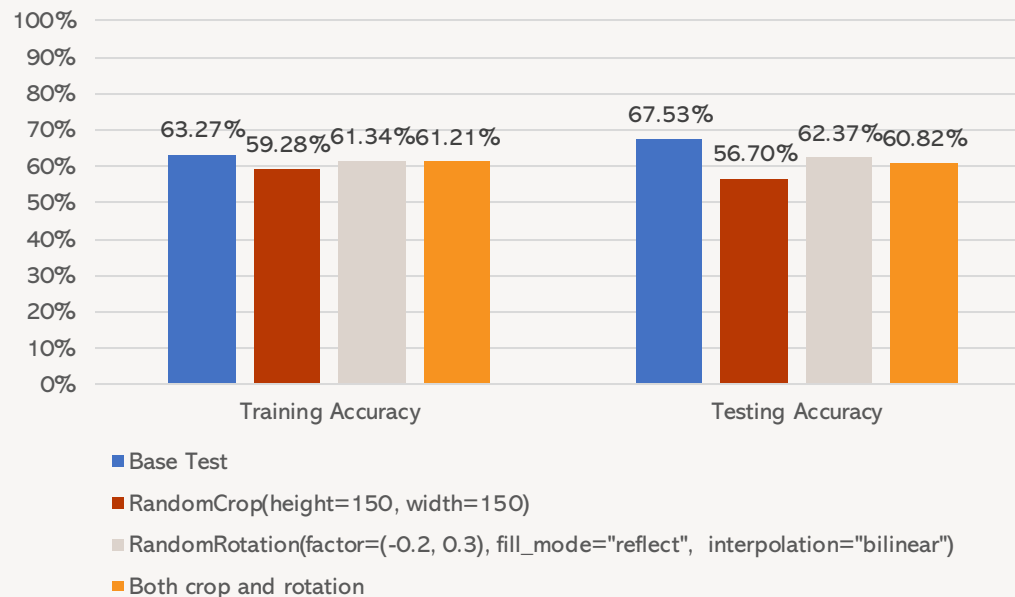


Method #3: Model Layers

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 173, 173, 16)	1048592
random_crop (RandomCrop)	(None, 150, 150, 16)	0
conv2d_1 (Conv2D)	(None, 87, 87, 64)	4194368
batch_normalization (Batch Normalization)	(None, 87, 87, 64)	256
conv2d_2 (Conv2D)	(None, 56, 56, 128)	8388736
max_pooling2d (MaxPooling2D)	(None, 28, 28, 128)	0
batch_normalization_1 (Batch Normalization)	(None, 28, 28, 128)	512
conv2d_3 (Conv2D)	(None, 13, 13, 256)	8388864
dropout (Dropout)	(None, 13, 13, 256)	0
flatten (Flatten)	(None, 43264)	0
dense (Dense)	(None, 1024)	44303360
dense_1 (Dense)	(None, 512)	524800
dense_2 (Dense)	(None, 256)	131328
dropout_1 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 128)	32896
dropout_2 (Dropout)	(None, 128)	0
dense_4 (Dense)	(None, 64)	8256
dense_5 (Dense)	(None, 32)	2080
dense_6 (Dense)	(None, 16)	528
dense_7 (Dense)	(None, 8)	136
dense_8 (Dense)	(None, 2)	18
Total params: 67,024,730		
Trainable params: 67,024,346		
Non-trainable params: 384		

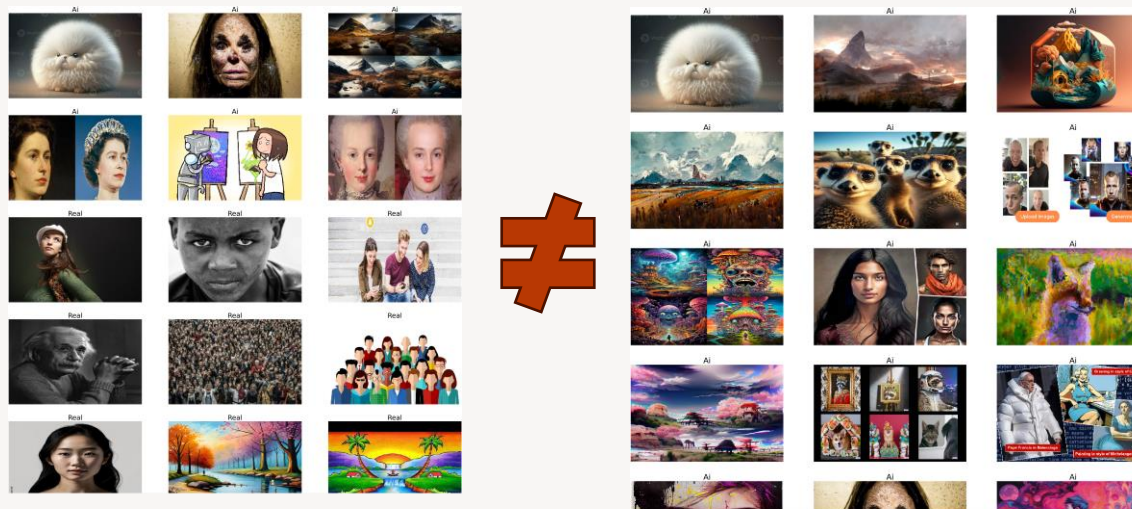
Training/Testing Accuracy for Transform Model Layer Tests



Overall, no improvements to the metrics like accuracy or F1 scores

Key takeaway – the Random Crop actually mispredicted more Real images compared to the typical trend of misclassifying AI images

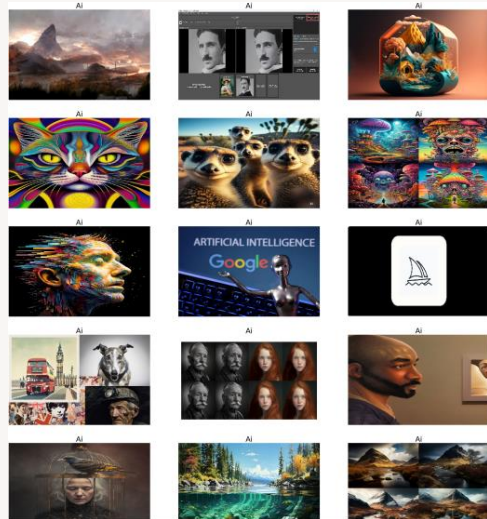
- Added image transform layers to the model at the beginning
- Epochs: 10
- learning_rate=0.01, decay = 0.05, momentum=0.3



Voting Test #1

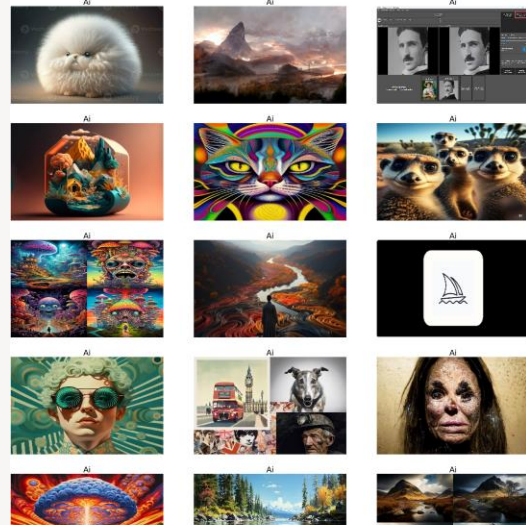
VGG16 Augmented All Model

Testing Accuracy: 76.28%



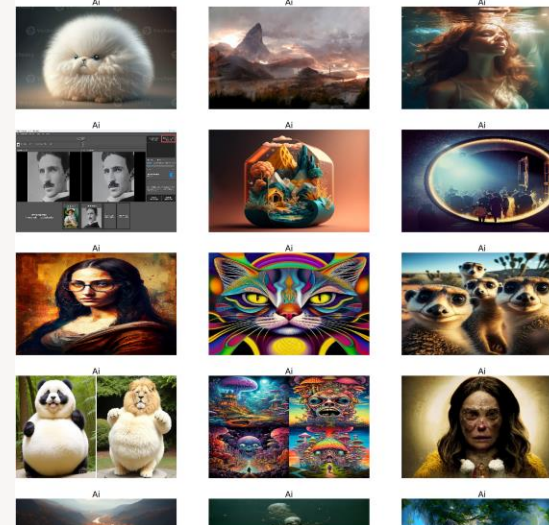
VGG16 Replace Brightness

Testing Accuracy: 76.80%



Zoom-only Augment

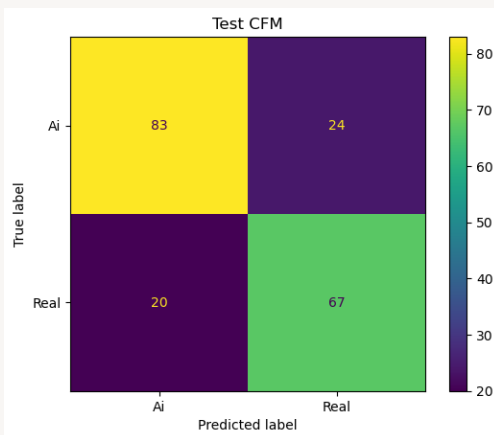
Testing Accuracy: 65.98%



About a 1% improvement to the results on best voting model → misclassified images still about the same

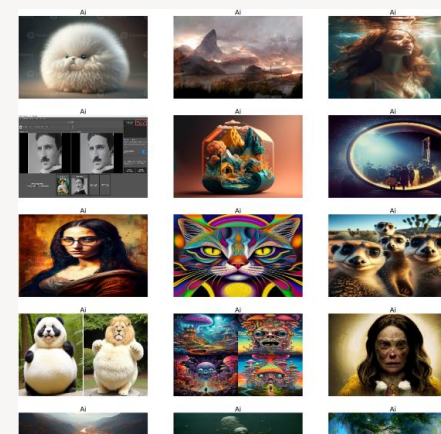
Seems like there are a lot of common images consistently mispredicted, would need models that have more drastic differences

If 2/3 vote 1 (Real) then real → Testing Accuracy: 77.32%



----Testing Metrics ----

	precision	recall	f1-score	support
0	0.81	0.78	0.79	107
1	0.74	0.77	0.75	87
accuracy			0.77	194
macro avg	0.77	0.77	0.77	194
weighted avg	0.77	0.77	0.77	194



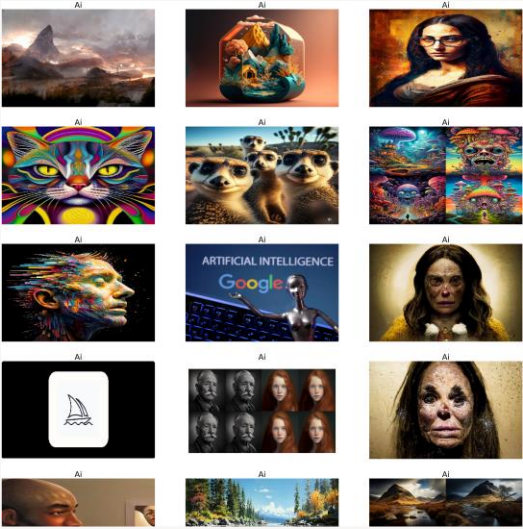
Voting Test #2

VGG16 Replace Bright (300epo)

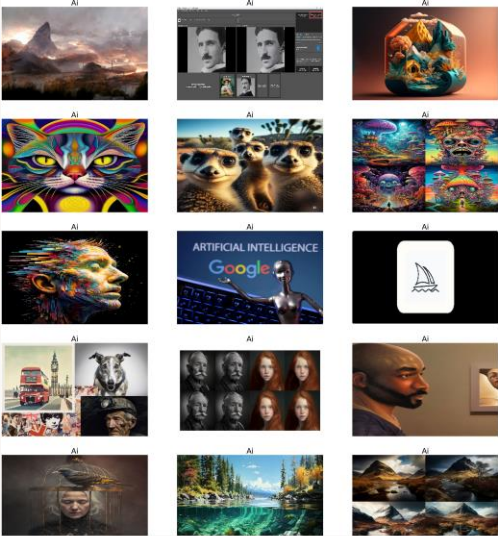
VGG16 Aug All (100epo)

Cropping Layer

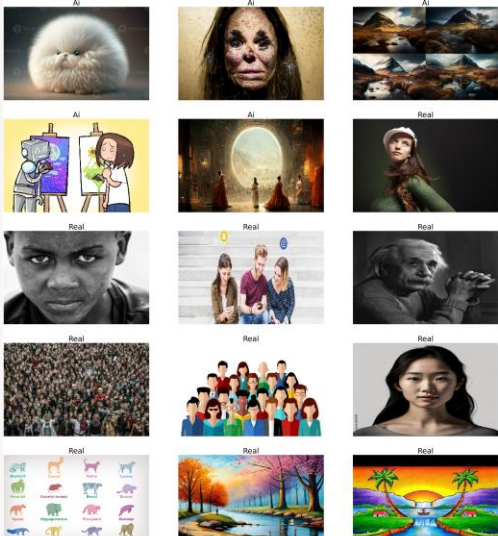
Testing Accuracy: 76.80%



Testing Accuracy: 76.29%



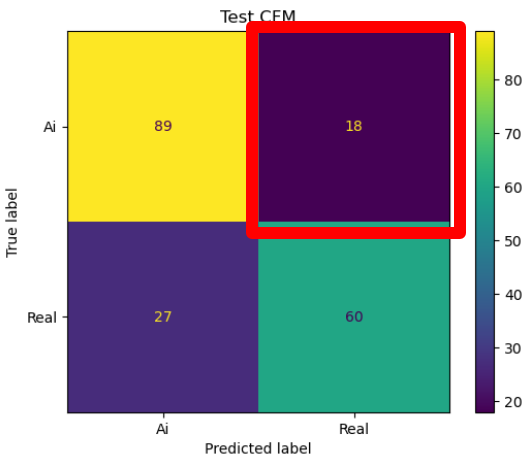
Testing Accuracy: 60.30%



No real improvements to the accuracy but could change which class was “favored” → applications where you would want more Real images misclassified as AI for later human review

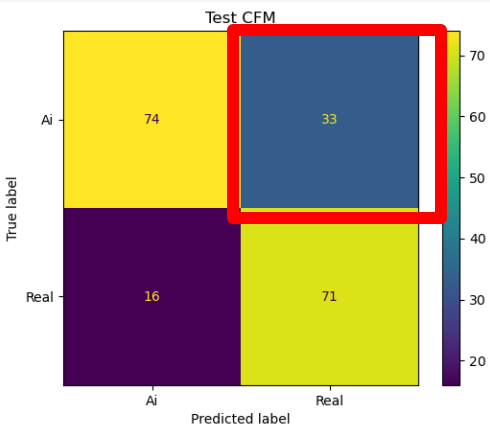
If 2/3 vote 1 (Real) then real→ 76.80 %

If 2/3 vote 1 (Real) then real→ 74.74%



----Testing Metrics ----

	precision	recall	f1-score	support
0	0.77	0.83	0.80	107
1	0.77	0.69	0.73	87
accuracy			0.77	194
macro avg	0.77	0.76	0.76	194
weighted avg	0.77	0.77	0.77	194



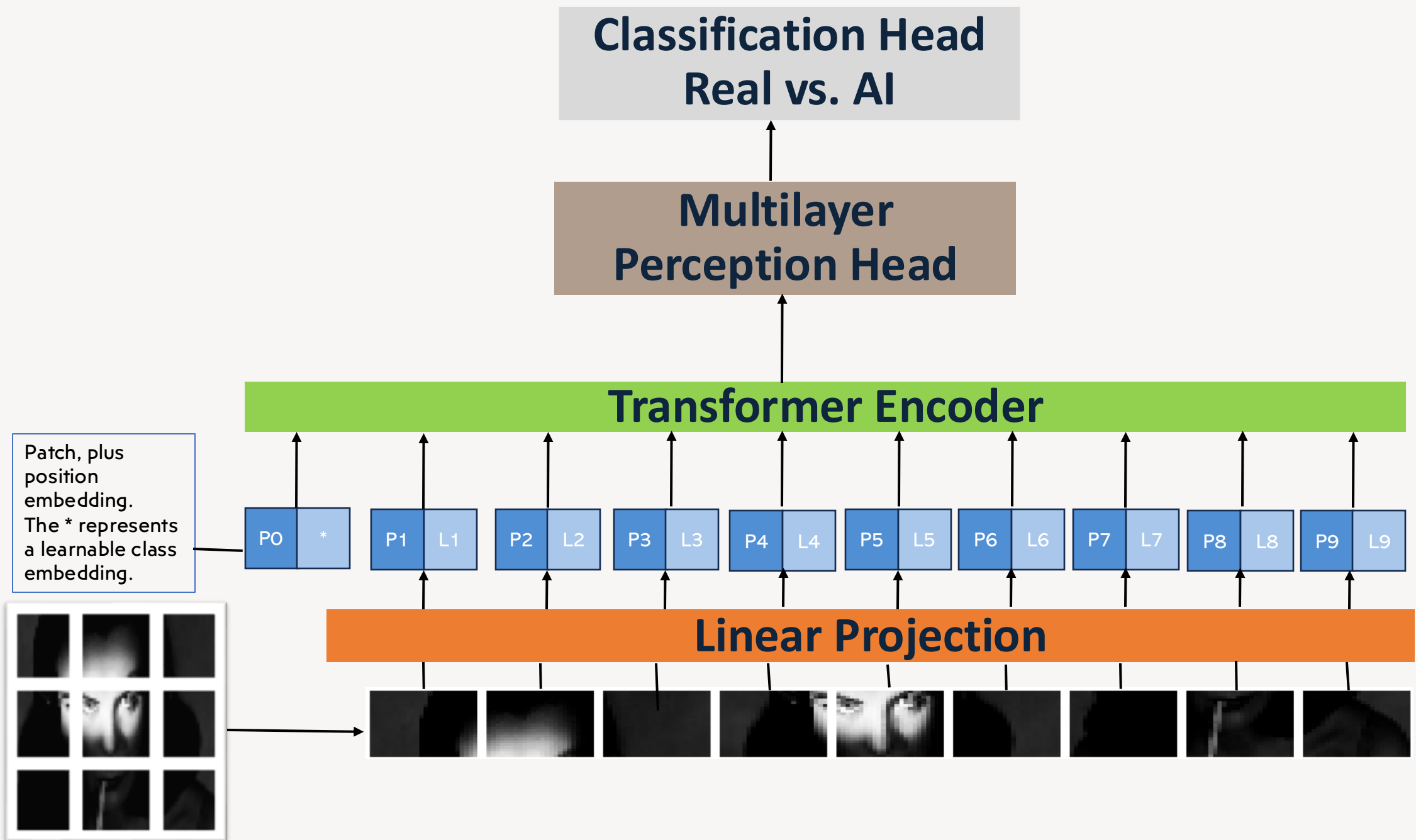
----Testing Metrics ----

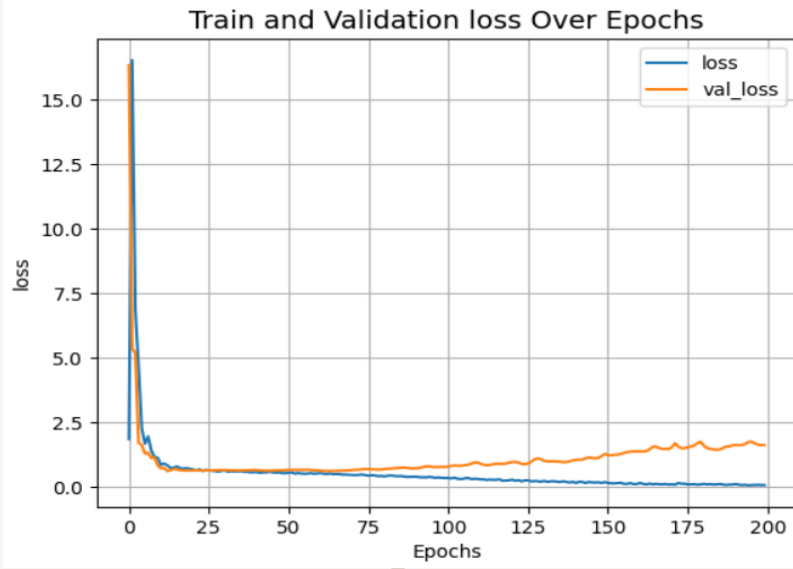
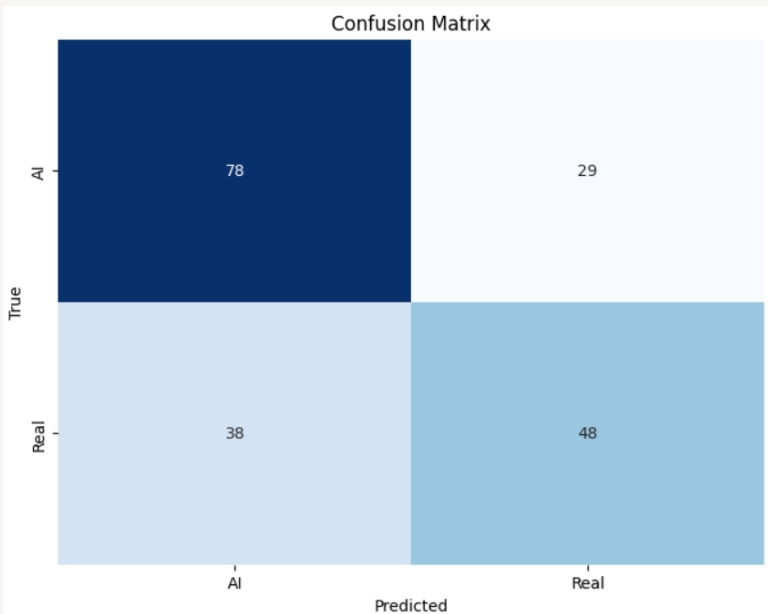
	precision	recall	f1-score	support
0	0.82	0.69	0.75	107
1	0.68	0.82	0.74	87
accuracy			0.75	194
macro avg	0.75	0.75	0.75	194
weighted avg	0.76	0.75	0.75	194



**ALTERNATIVE
TO CNN**

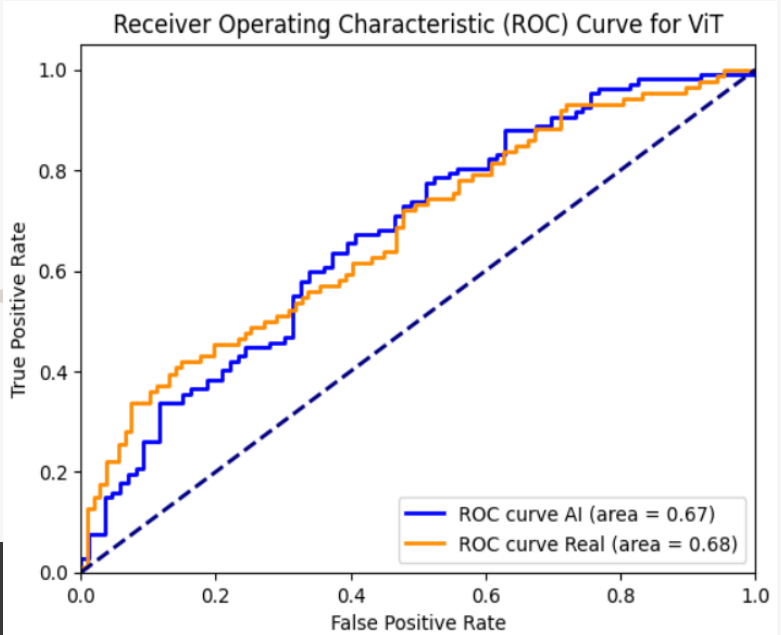
Vision Transformers (ViT)

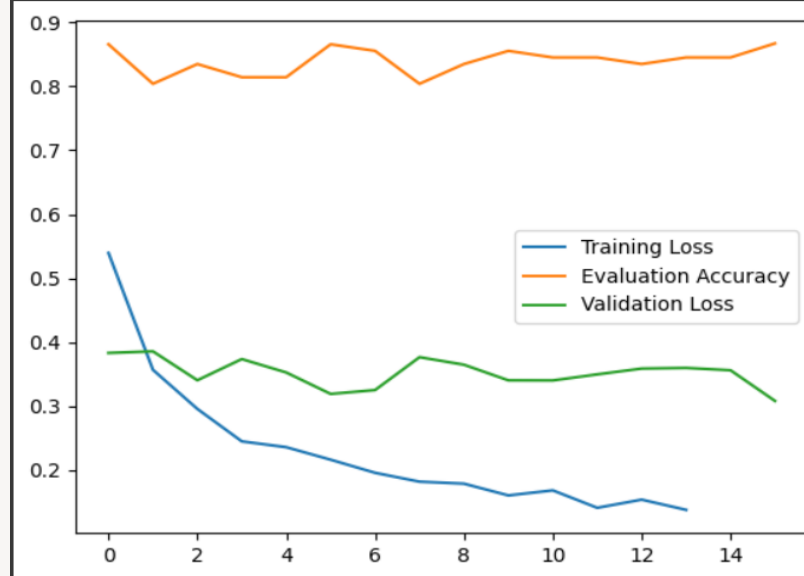
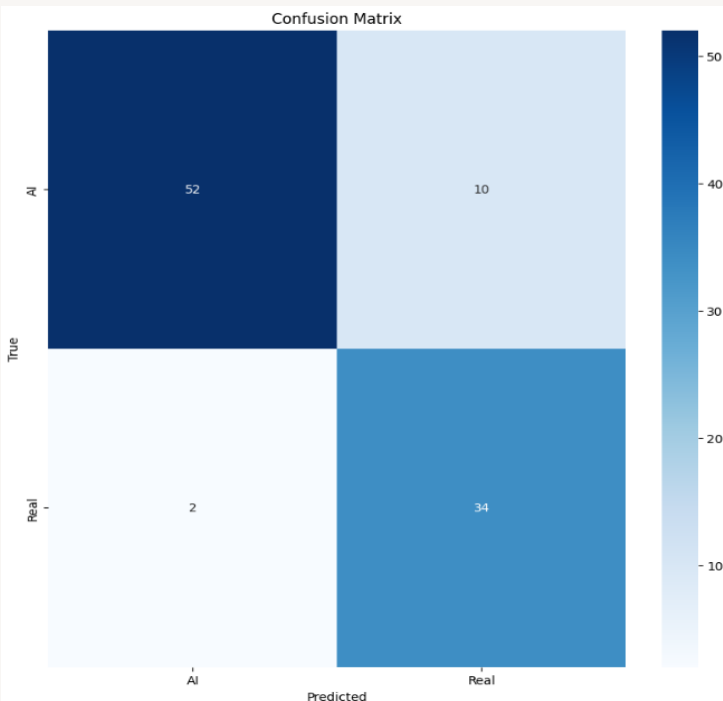




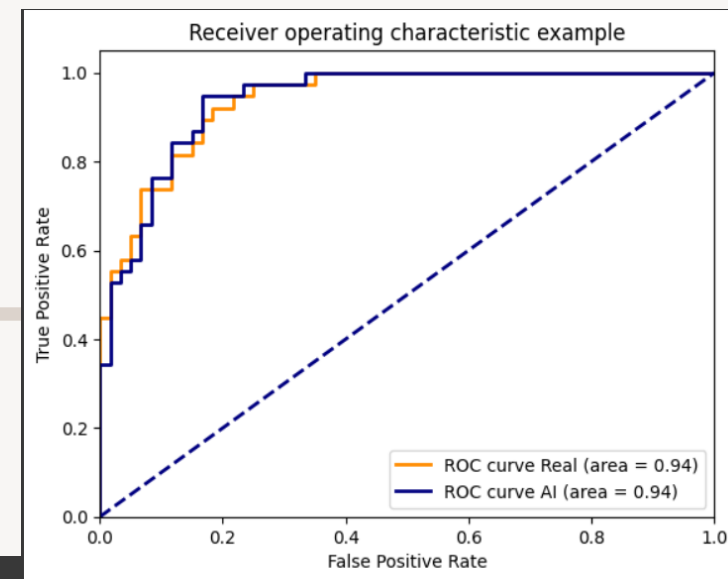
From Scratch ViT

Classification report:					
		precision	recall	f1-score	support
	0.0	0.67	0.73	0.70	107
	1.0	0.62	0.56	0.59	86
	accuracy			0.65	193
	macro avg	0.65	0.64	0.64	193
	weighted avg	0.65	0.65	0.65	193





Pretrained ViT (Google patch16)



Classification report:

	precision	recall	f1-score	support
0.0	0.90	0.84	0.87	107
1.0	0.82	0.88	0.85	86
accuracy			0.86	193
macro avg	0.86	0.86	0.86	193
weighted avg	0.86	0.86	0.86	193

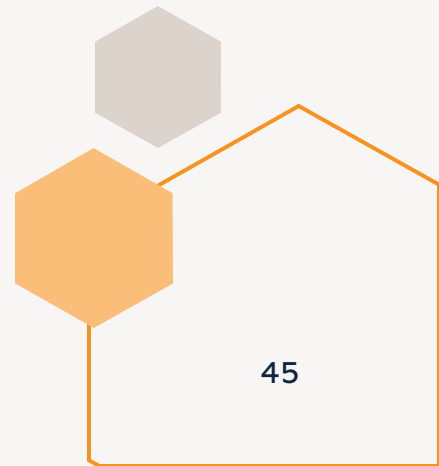
**REAL WORLD
USAGE**



Application

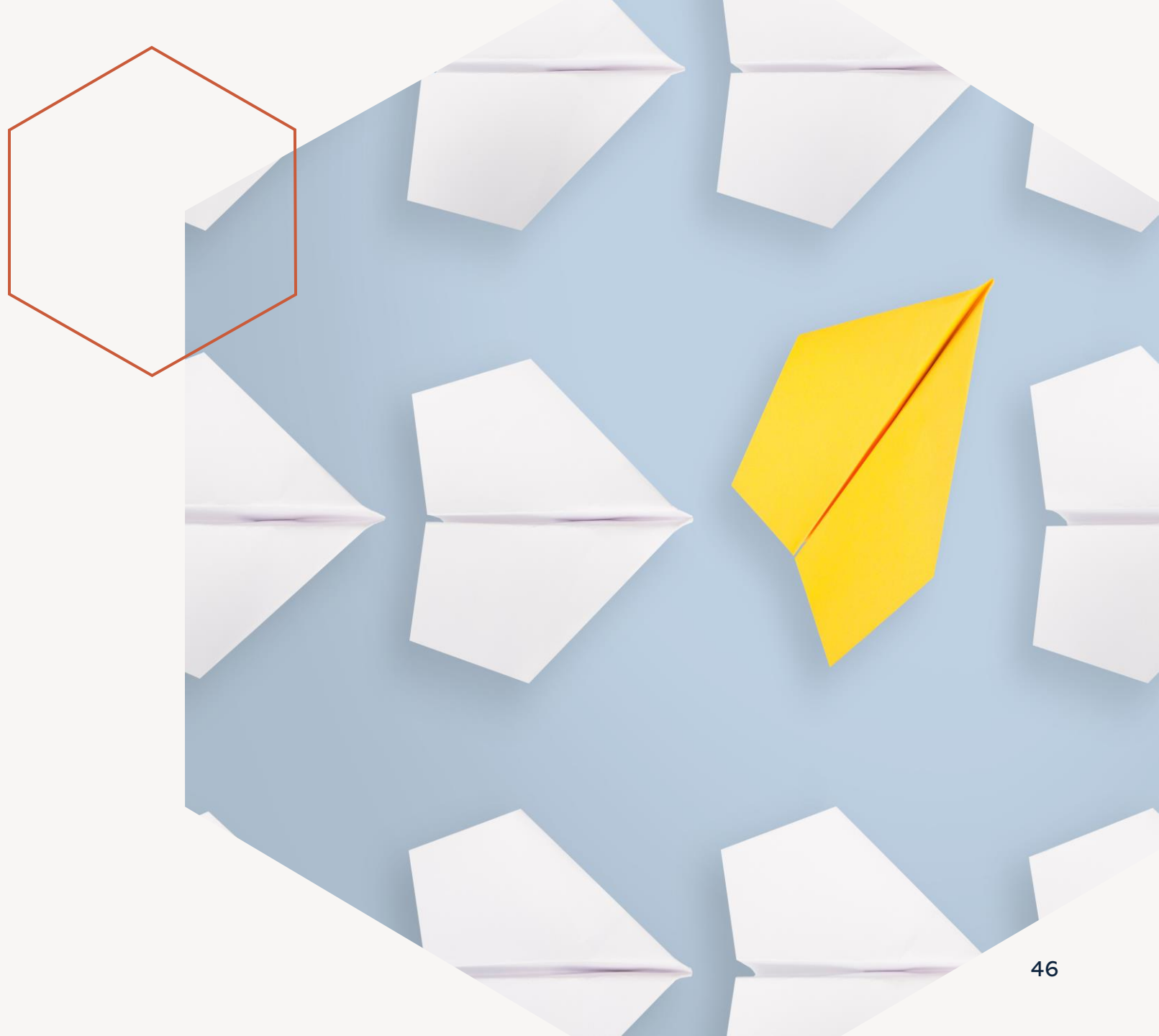
Live Demo

- Screenshare of application



Summary

Key takeaways of all experiments



Key Takeaways



Best Results

Key contributors

- ViT using Google's pretrained had the overall best results
- Pre-trained models in general significantly performed better than any models built from scratch

Tests

Key summaries

- Augmentation/transforms didn't necessarily improve the metrics but did change what types of images were mispredicted

Challenges

Post-Review of Tests

- Certain images were commonly mispredicted (i.e. ones with real-ai side-by-side)
- Model training limited by GPU space on both Collab & 3070ti



**Thank you for
listening**